Overview

A. Industry applications of image recognition

Image recognition is one of the most important tasks in industry machine learning, specifically in computer vision. Computer vision, the discipline dealing with how computers can gain high-level understanding of images and videos, powers everything from self-driving cars to facial identification. Many companies, including giants like Uber and Facebook, have teams dedicated to computer vision research and application.

How image recognition works

Images can be thought of as arrays of pixels. In most cases, each pixel can be viewed as a set of three numbers, corresponding to the pixel's RGB values. So the first process of image recognition is to process all the image data into arrays containing the numeric RGB values for each pixel.

After processing the images, we feed the data arrays into a convolutional neural network. A convolutional neural network (CNN) is a type of neural network that works extremely well for image-related tasks. You'll learn about the CNN architecture and different variations of CNNs in this course.

The CNN will apply various computations to the input image data, producing a probability (or set of probabilities for multiclass classification). The class with the highest probability is the class that the CNN categorizes the image into. For more on classification using neural networks, check out the Machine Learning for Software Engineers course.

C. What will this course provide

After taking this course, you'll be able to process image data, train a CNN on the image data, and use the CNN to perform image recognition. Specifically, you will be able to:

- Take raw images and process them into usable data for machine learning. This includes techniques such as image standardization, image resizing, and data augmentation.
- Build a CNN from scratch and different variations of the CNN such as SqueezeNet and ResNet.
- Use a CNN to perform different types of image recognition, from digit recognition to classifying the ImageNet dataset.

Introduction

In this section you will learn about image processing with **TensorFlow** and **PIL**. Before doing any image-related tasks, we need to convert the images into a usable form for deep learning models. TensorFlow and PIL provide us utilities that make image processing very straightforward.

A. Image recognition

Computer vision, the field concerning machines being able to understand images and videos, is one of the hottest topics in the tech industry. Robotics, self-driving cars, and facial recognition all rely on computer vision to work. At the core of computer vision is *image recognition*, the task of recognizing what an image represents.

Before performing any task related to images, it is almost always necessary to first process the images to make them more suitable as input data. This section of the course focuses on image processing, specifically how we can convert images from JPEG or PNG files to usable data for our neural networks.

B. Core libraries

The two main libraries that will be used in this Lab for image processing are TensorFlow and PIL. TensorFlow provides a variety of utility functions to obtain image data from files, resize the images, and even transform a large set of images all at once. The original PIL (Python Imaging Library) is an image manipulation library built for Python. Although its development stopped in 2011, an offshoot called Pillow was created and continues to be developed to provide better image manipulation tools for Python users. We'll use the Pillow version of PIL for more fine-grain image processing.

Image Files

Discover how image data is stored in files and how pixels affect image interpretation.

Chapter Goals:

- Learn about image data and pixels
- Read byte data from an image file

A. Image data

Before we do any image processing, we need to understand how image files work. Specifically, we'll discuss how these files use byte data and pixels to represent images.

If you've ever looked at an image file's properties before, it'll show the dimensions of the image, i.e. the height and width of the image. The height and width are based on number of pixels. For example, if the dimensions of an image are 400x300 (width x height), then the total number of pixels in the image is 120000.

The basketball image has dimensions 200x200 (200px width, 200px height), while the tennis ball image is 72x66 (72px width, 66px height).

The function `tf.read_file` takes the file name as its required argument and returns the contents of the file as a tensor with type `tf.string`. When the input file is an image, the output of `tf.read_file` will be the raw byte data of the image file. Although the raw byte output represents the image's pixel data

the image file. Although the raw byte output represents the image's pixel data,

it cannot be used directly. In the next chapter we'll see how we can convert the raw data to usable pixel data.

B. Pixels

So what exactly is a pixel? A pixel is essentially just a point on an image, with a specific shade, color, and/or opacity. We normally represent a pixel as a single integer or multiple integers. Pixels take a specific form based on the interpretation of the image, which is usually one of the following:

- ***Grayscale***: Viewing the image as shades of black and white. Each pixel is an integer between 0-255, where 0 is completely black and 255 is completely white.
- ***RGB***: The default interpretation for color images. Each pixel is made up of 3 integers between 0-255, where the integers represent the intensity of red, green, and blue, respectively, for the pixel.
- ***RGBA***: An extension of RGB with an added *alpha* field. The alpha field represents the opacity of an image, and in this Lab we'll represent a pixel's alpha value as an integer from 0-255 with 0 being fully transparent and 255 being fully opaque.

We can choose to interpret an image however we want, but there is usually one interpretation that is optimal. For example, we could interpret a black and white image with RGB pixel values, but it is more efficient to view it as a grayscale image (3x fewer integers used). On the other hand, it would be unwise to interpret a colored image using grayscale pixels, since the pixels won't be able to capture any of the actual colors.

Time to Code!

In the next three chapters you'll be working on the `decode_image` function, which decodes image data from a file.

In this chapter, we'll use `tf.read_file` to read the image binary data from the file.

Set `value` equal to the output of `tf.read_file` with argument `filename`.

```
import tensorflow as tf

# Decode image data from a file in Tensorflow

def decode_image(filename, image_type, resize_shape, channels=0):
    # CODE HERE
    pass
```

Image Types

Learn how to decode raw image data into pixel data using TensorFlow.

Chapter Goals:

- Decode raw byte data into usable pixel data
- Understand the relationship between channels and pixels

A. Decoding

We'll now decode the raw byte data into usable pixel data. The decoding function that we use depends on the format of the image. If the input is a PNG image then we use `tf.image.decode_png`, and if the input is a JPEG image we use `tf.image.decode_jpeg`. For generic decoding (i.e. decoding any image format), we use `tf.image.decode_image`.

Since `tf.image.decode_image` can decode any type of image, you might be wondering why we even bother with the other two decoding functions. One reason is that you may want to only use specific image formats, in which case it's more efficient and better for code clarity to just use the format-specific decoding function.

Another reason is that `tf.image.decode_image` supports GIF decoding, which results in an output shape of `(num_frames, height, width, channels)`. Since the function can return data with different shapes, we can't use `tf.image.decode_image` when we also need to resize the image with `tf.image.resize_images` (see next chapter).

B. Channels

In the previous chapter we discussed image interpretations. If we only pass in `value` as the required argument for one of the decoding functions, we're using the interpretation specified in the raw image data. Normally it's fine to do this, but sometimes we want to use a specific format for the pixels.

We can change the pixel format of the decoded image via the `channels`

keyword argument.

```
import tensorflow as tf
value = tf.read_file('image3.jpg')
with tf.Session() as sess:
    arr = sess.run(tf.image.decode_jpeg(value, channels=1))
    print(arr.shape)
    print(repr(arr))
```

As you can see, the `channels` argument represents the number of integers per pixel. The default value for `channels` is `0`, which means the decoding function uses the interpretation specified from the raw data. Setting `channels` to `1` specifies a grayscale image, while setting `channels` to `3` specifies an RGB image. For PNG images we're also allowed to set `channels` to `4`, corresponding to RGBA images. Setting `channels` to `2` is invalid.

Time to Code!

In this chapter we'll continue to work on the `decode_image` function from the previous chapter. The previous chapter's code is already filled in the function.

We decode `value` based on the image type. There are two image types, PNG and JPEG, that we will specifically check for.

Create an `if...elif...else` code block, where the `if` condition checks that `image_type` equals `'png'` and the `elif` condition checks that `image_type` equals `'jpeg'`.

If the image type is PNG or JPEG we'll use the specific decoding function for the type. For all other image types (e.g. gif) or if the image type is unknown (`image_type=None`), we'll use the generic image decoding function.

Inside the `if` block, set `decoded_image` equal to the output of `tf.image.decode_png` with required argument `value` and keyword argument `channels=channels`.

Inside the `elif` block, set `decoded_image` equal to the output of `tf.image.decode_jpeg` with required argument `value` and keyword argument `channels=channels`.

Inside the `else` block, set `decoded_image` equal to the output of `tf.image.decode_image` with required argument `value` and keyword argument `channels=channels`.

```
import tensorflow as tf

# Decode image data from a file in Tensorflow
def decode_image(filename, image_type, resize_shape, channels=0):
    value = tf.read_file(filename)
    # CODE HERE
```

Resizing

Use TensorFlow to resize images with a variety of different image scaling algorithms.

Chapter Goals:

- Be able to resize pixel data when required
- Understand how resizing works in TensorFlow

A. Basic resizing

The function we use for resizing pixel data is `tf.image.resize_images`. It takes in two required arguments: the original image's decoded data and the new size of the image, which is a tuple/list of two integers representing `new_height` and `new_width`, in that order.

```
import tensorflow as tf

with tf.Session() as sess:
    print('Original: {}'.format(
        repr(sess.run(decoded_image))))  # Decoded image data
    resized_img = tf.image.resize_images(decoded_image, (3, 2))
    print('Resized: {}'.format(
        repr(sess.run(resized_img))))
```

Example usage of tf.image.resize_images. Note that the resized pixel array has type float, and the number of channels is still 1.

The function compresses or expands the image (depending on the relationship between the new image dimensions and the old image dimensions) and then returns the pixel data for the resized image, with the same number of channels. Note that if the resized dimensions don't match the same aspect ratio as the original dimensions, the new image data will be distorted.

In the example above, the original image's dimensions were 3x3, while the resized dimensions were 3x2. This resulted in the image data becoming distorted.

distorted.

B. Resizing methods

The TensorFlow `tf.image.resize_images` function allows us to specify a keyword argument called `method`. The `method` argument represents the image scaling algorithm. There are 4 possible values for `method`:

- `tf.image.ResizeMethod.BILINEAR`
- `tf.image.ResizeMethod.NEAREST_NEIGHBOR`
- `tf.image.ResizeMethod.BICUBIC`
- `tf.image.ResizeMethod.AREA`

The default value for `method` is `tf.image.ResizeMethod.BILINEAR`. A nice comparison of some of the methods can be found here. The comparison does not mention the `AREA` method, which is normally used for downsampling (resizing to a smaller size).

C. Unknown type

As mentioned in the previous chapter, a benefit to using `tf.image.decode_image` is when we don't know the type of input image (e.g. PNG vs. JPEG). However, we can't use `tf.image.resize_images` if the decoding function was `tf.image.decode_image`. This is because the input data for `tf.image.resize_images` needs to have a known number of dimensions, but the output of `tf.image.decode_image` can have 3 or 4 dimensions depending on the image type.

If it is still necessary to resize an image of unknown type (non-GIF), we can use `tf.image.resize_image_with_crop_or_pad`. This resizes pixel data by either padding the data with 0's (for a size increase) or cropping the pixel data (for a size decrease). Cropping the pixel data means removing certain pixels along each dimension that needs to be shrunk.

In contrast to `tf.image.resize_images`, the output of `tf.image.resize_image_with_crop_or_pad` is the same type as the original image data, since none of the individual pixels are transformed.

```
import tensorflow as tf

with tf.Session() as sess:
```

```
    print('Original: {}'.format(
        repr(sess.run(decoded_image))))  # Decoded image data
    resized_img = tf.image.resize_image_with_crop_or_pad(

        decoded_image, 5, 2)
    print('Resized: {}'.format(
        repr(sess.run(resized_img))))
```

Example usage of tf.image.resize_image_with_crop_or_pad. Note that the resized pixel array remains the same type.

In the example, we resize a 4x3 image (with 1 channel) to new dimensions of 5x2. The second argument of `tf.image.resize_image_with_crop_or_pad` represents the new height, while the third argument represents the new width.

To decrease the width and increase the height, cropping is applied along the width dimension, while padding is applied along the height dimension.

Time to Code!

In this chapter we'll be completing the `decode_image` function.

We need to check that there is a specified `resize_shape` and the `image_type` is valid.

Create an `if` code block outside the previous `if...elif...else` block. The `if` condition checks that both `resize_shape` is not `None` and `image_type` is either `'png'` or `'jpeg'`.

If the previous conditions are met, then we can resize the decoded image.

Inside the `if` block, set `decoded_image` equal to the output of `tf.image.resize_images` with first argument `decoded_image` and second argument `resize_shape`.

To finish the function we'll return `decoded_image`, outside the scope of the `if` code block. This ensures that the pixel data is returned regardless of whether it is resized or not.

Return `decoded_image`, outside the scope of the `if` block.

```
import tensorflow as tf

# Decode image data from a file in Tensorflow

def decode_image(filename, image_type, resize_shape, channels=0):
    value = tf.read_file(filename)
    if image_type == 'png':
        decoded_image = tf.image.decode_png(value, channels=channels)
    elif image_type == 'jpeg':
        decoded_image = tf.image.decode_jpeg(value, channels=channels)
    else:
        decoded_image = tf.image.decode_image(value, channels=channels)
    # CODE HERE
```

Dataset

Create a dataset of image files and convert them into pixel data.

Chapter Goals:

- Learn how to create a dataset of image files
- Convert a dataset of image files into decoded pixel data

A. Image dataset

Normally when we do image related tasks we're dealing with a large amount of image data. In this case, it's best to use a TensorFlow dataset, i.e. `tf.data.Dataset`, to store all the images. We can create a dataset using the `from_tensor_slices` function.

The `Dataset` class makes it easier and more efficient to perform tasks with all the image files. If you want to learn more about using TensorFlow's `tf.data` API, check out the Industry Case Study course on Educative.

B. Mapping

After we create a dataset with the image files, we will need to decode each file's contents into usable pixel data. Since the `decode_image` function works for single image files, we will need to use the dataset object's `map` function to apply `decode_image` to each image file in our dataset.

The output of the `map` function is a new dataset with each element now converted from the original image file to its corresponding pixel data. We use `map` rather than using a `for` loop to manually convert each image file because `map` does the image decoding in parallel across the files, making it a more efficient solution.

```
import tensorflow as tf

image_paths = ['img1.jpg', 'img2.jpg']
dataset = tf.data.Dataset.from_tensor_slices(image_paths)
def _map_fn(filename):
```

```
    # FUNCTION FROM PREVIOUS CHAPTERS
    return decode_image(...)
map_dataset = dataset.map(_map_fn)
```

Dataset of image files converted to a dataset of pixel data using the map function.

In the example above, `dataset` represents a dataset containing the names of each file. After we apply the `map` function, the output dataset (`map_dataset`) contains the pixel data from each image file.

Time to Code!

In this chapter we'll be creating a function called `get_dataset`, which creates a dataset of image data from file paths.

We'll first convert the image file paths to a tensor of strings, so we can process it as a dataset.

Set `filename_tensor` equal to `tf.constant` with argument `image_paths`.

Now we can create our dataset from the tensor of image file paths.

Set `dataset` equal to `tf.data.Dataset.from_tensor_slices` with argument `filename_tensor`.

To apply our image decoding to each image file in the dataset, we need a wrapper function. Our wrapper function will be created **within the scope of `get_dataset`**.

Right below the initialization of `dataset`, define a function called `_map_fn` with a single required argument, `filename`.

Inside `_map_fn`, return `decode_image` applied with required arguments `filename`, `image_type`, and `resize_shape`, as well as keyword argument `channels=channels`.

We can now apply the mapping wrapper to our dataset, and return the output.

Return `dataset.map` applied with argument `_map_fn`.

```
import tensorflow as tf

def decode_image(filename, image_type, resize_shape, channels=0):
    value = tf.read_file(filename)
```

```
    if image_type == 'png':
        decoded_image = tf.image.decode_png(value, channels=channels)
    elif image_type == 'jpeg':
        decoded_image = tf.image.decode_jpeg(value, channels=channels)
    else:
        decoded_image = tf.image.decode_image(value, channels=channels)
    if resize_shape is not None and image_type in ['png', 'jpeg']:
        decoded_image = tf.image.resize_images(decoded_image, resize_shape)
    return decoded_image

# Return a dataset created from the image file paths
def get_dataset(image_paths, image_type, resize_shape, channels):
    # CODE HERE
    pass
```

Iterator

Set up an Iterator to extract data from an image dataset.

Chapter Goals:

- Set up an `Iterator` to extract data from a pixel array dataset

A. Using Iterator

The way we can extract the decoded image data from our `Dataset` is through a `tf.data.Iterator`. For a more in-depth look at `tf.data.Iterator`, check out the Industry Case Study course on Educative.

We use the `get_next` function to obtain a *next-element tensor*, which is used for data extraction. Note that the next-element tensor doesn't have an actual value until we execute the iteration process using `tf.Session` (see next chapter).

Time to Code!

In this chapter you'll be working on the `get_image_data` function. This function uses an `Iterator` object to get decoded image data from a dataset.

Using the `get_dataset` function from the previous chapter (not shown), we can create our image pixel `Dataset`.

Set `dataset` equal to `get_dataset` with arguments `image_paths`, `image_type`, `resize_shape`, and `channels`.

We'll now make an `Iterator` for `dataset`.

Set `iterator` equal to `dataset.make_one_shot_iterator` with no arguments.

The one-shot iterator is a very simple iterator. It is associated with a particular dataset and only iterates through it once.

Finally, we set up the next-element tensor for extracting data from `dataset`.

Set `next_image` equal to `iterator.get_next` with no arguments.

```
import tensorflow as tf

# Get the decoded image data from the input image file paths
def get_image_data(image_paths, image_type=None, resize_shape=None, channels=0):
    # CODE HERE
    pass
```

Execution

Learn how to execute data extraction in TensorFlow.

Chapter Goals:

- Iterate through a dataset and extract pixel data using `tf.Session`

A. Data iteration

In the previous chapter we set up a next-element tensor using the `get_next` function. The way we actually execute the data extraction is by using the `run` function of `tf.Session`.

Each time we use `sess.run(next_image)`, we are iterating a single step through our dataset. So the first time we use `sess.run(next_image)`, it'll return the first pixel array in `dataset` (as a NumPy array), and the ith time it'll return the ith pixel array in `dataset`.

```
it = dataset.make_one_shot_iterator()
next_image = it.get_next()
sess = tf.Session()
for i in range(3):
    sess.run(next_image)
```

Iteration through a dataset. Each time sess.run(next_image) is called, it returns the dataset value at the current iteration.

We can only call `sess.run(next_image)` for the number of images there are in the dataset before we get an `OutOfRangeError`.

B. Using data

Rather than going through the decoding process every time we use a dataset of image files, it is usually better to just decode once and save the NumPy pixel data in a file. This can be done with the `numpy.save` function.

The code below shows examples of using `numpy.save`. The file itself is not human-readable, but can be loaded back into a NumPy array with `numpy.load`

```
import numpy as np

arr = np.array([1, 2, 3])
# Saves to 'arr.npy'
np.save('arr.npy', arr)
# Also saves to 'arr.npy'
np.save('arr', arr)
# Loading from that file
arr_copy = np.load('arr.npy')
print(repr(arr_copy))
```

There are other more efficient ways to store the decoded data. However, for smaller projects that don't use too large of an image dataset, the way of saving NumPy data in a file works fine.

Time to Code!

In this chapter we'll be completing the `get_image_data` function from the previous chapter.

Since we're going to return a list of all our image pixel data, we want to first initialize this list.

Set `image_data_list` equal to an empty list.

We'll put our iteration and execution code within the scope of a `tf.Session`. For more information on `tf.Session` scopes, see the Machine Learning for Software Engineers course on Educative.

Create a `with tf.Session() as sess` code block.

Inside the scope of `sess`, create a `for` loop that iterates `i` through `range(len(image_paths))`.

Inside the `for` loop, we'll execute a single data extraction and append the extracted pixel array to the list.

Set `image_data` equal to `sess.run(next_image)`.

Append the resultant `image_data` to `image_data_list`.

After iterating through the entire dataset, we can return `image_data_list`.

Outside the scope of `sess`, return `image_data_list`.

```
import tensorflow as tf

# Get the decoded image data from the input image file paths
def get_image_data(image_paths, image_type=None, resize_shape=None, channels=0):
    dataset = get_dataset(image_paths, image_type, resize_shape, channels)
    iterator = dataset.make_one_shot_iterator()
    next_image = iterator.get_next()
    # CODE HERE
```

PIL Library

Use the PIL library to extract and modify data from an image.

Chapter Goals:

- Learn the basics of the `PIL` module
- Use `PIL` to load an image and return its resized pixel data

A. PIL module

While we can do large scale image processing in TensorFlow, the `PIL` module (from Pillow) allows us to do more fine-grained image processing. In this Lab we will demonstrate basic resizing and filtering as an introduction to `PIL`. However, the library also has many more utility functions for advanced image processing and analysis. Documentation and extended examples using the `PIL` module can be found here.

The `PIL` module has a submodule called `Image`, which is the main module used for image processing. With it, we can create an `Image` object from an image file and obtain the image's pixel data. The nice thing about PIL is that the methods are more user-friendly than the TensorFlow methods; for example, the file reading and decoding are done together and resizing can work with unknown image formats.

The `image_mode` variable refers to the interpretation of the pixels, e.g. Grayscale vs. RGB vs. RGBA. PIL provides us with many types of modes.

We'll use `image_mode` to convert our image into RGBA format so that we can resize it without worrying about what format it's in.

B. Image modification

Similar to TensorFlow, the `Image` module allows us to resize our image. Also like TensorFlow, the `Image` module's resizing function takes in an optional resizing method, which the `PIL` documentation refers to as *resampling filters*. Detailed descriptions, as well as a scaling quality and performance

comparison for each resampling filter can be found here. In the code at the

end of this chapter, we will use the Lanczos resampling filter for the best quality resizing.

`PIL` also allows us to apply *filtering* to an image, which is provided via the `ImageFilter` submodule. The `ImageFilter` module has a substantial list of predefined filters, as well as some more advanced filter classes.

Filtering allows us to perform tasks such as sharpening or blurring an image's features. Filters like these are actually the crux of image recognition, and they're discussed in more detail in the **CNN** section of this course.

Time to Code!

In this chapter we'll be completing the `pil_resize_image` function, which loads and resizes an image using `PIL`.

We'll first load the image using the `Image` module, and convert it to the specified `image_mode`.

Set `im` equal to `Image.open` applied to `image_path`.

Set `converted_im` equal to `im.convert` applied to `image_mode`.

After converting the image, we'll resize it using Lanczos filtering.

Set `resized_im` equal to `converted_im.resize` applied with first argument `resize_shape` and second argument `Image.LANCZOS`.

Before returning our image data, we'll check to see if an `image_filter` is specified.

Create an `if` code block which checks that `image_filter` is not `None`.

If an `image_filter` is specified, apply it to `resized_im`.

Inside the `if` block, set `resized_im` equal to `resized_im.filter` applied with argument `image_filter`.

Now we can return the image pixel data, converted to a NumPy array.

Outside the `if` block, set `im_data` equal to the output of

`resized_im.getdata` applied with no arguments.

Then return the output of `np.asarray` applied with `im_data` as the only argument.

```
import numpy as np
from PIL import Image, ImageFilter

# Load and resize an image using PIL, and return its pixel data
def pil_resize_image(image_path, resize_shape,
    image_mode='RGBA', image_filter=None):
    # CODE HERE
    pass
```

Quiz

1 How would we distinguish between grayscale and RGB image data?

COMPLETED 0%

1 of 3

Introduction

In this section of the course you will be writing a basic convolutional neural network (CNN) for handwritten digit recognition of the MNIST dataset. We will take an in-depth look at a model based off of Yann LeCun's LeNet-5.

A. Digit recognition

When we start building an image recognition model for the first time, it's usually a good idea to train and evaluate it on a relatively simple dataset. This allows for less complicated models, quicker training, and less image preprocessing. Basically, we want to learn the inner workings of a convolutional neural network before we even try to apply it to more complex tasks.

One of the simplest tasks we can perform is handwritten digit recognition. Given an image of a handwritten digit (i.e., 0, 1, ..., 9), we want our model to be able to correctly classify its numeric value. Though this task seems relatively simple, it is actually used fairly often in real life, such as automatically extracting credit card numbers from a picture. The dataset we will use for digit recognition is the MNIST dataset, which is the de facto dataset used for machine learning-based digit recognition.

B. CNN dominance

Nowadays, every task in the field of image recognition is dominated by convolutional neural networks (CNNs). Since the introduction of LeNet-5, the pioneering CNN, in 1998 for digit recognition, CNN architectures have become king in nearly all computer vision tasks. The CNN forms the backbone of many of today's cutting-edge technologies, such as self-driving cars, robotics, and facial recognition.

Although CNNs dominate image related tasks, we could still theoretically use a multilayer perceptron model for classifying images. However, the MLP would

not perform nearly as well as a CNN. So what exactly makes a CNN special for image input data? The key to the model's success is its *convolutional layers*, which is one of the main topics we'll discuss in the following chapters.

Initialization

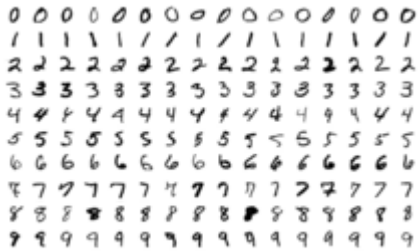
Learn about the MNIST dataset along with the input and output sizes for the data.

Chapter Goals:

- Learn about the MNIST database
- Initialize the model with input and output size

A. MNIST

The MNIST (Modified National Institute of Standards and Technology) database contains 60,000 training examples and 10,000 testing examples. The database contains grayscale handwritten digits that were resized to fit in a 20x20 pixel box, which was then centered in a 28x28 image (padded with whitespace). The images were resized using an anti-aliasing technique to minimize image distortion.



Examples from the MNIST database.

The MNIST database is available on Yann LeCun's website for download. The database is normalized to have floating point values between 0.0 and 1.0. In this case, 0.0 corresponds to a grayscale pixel value of 255 (pure white), while 1.0 corresponds to a grayscale pixel value of 0 (pure black).

B. Inputs and labels

Since each grayscale image has dimensions 28x28, there are 784 pixels per image. Therefore, each input image corresponds to a tensor of 784 normalized floating point values between 0.0 and 1.0. The label for an image is a one-hot tensor with 10 classes (each class represents a digit). In terms of our code, we

tensor with 10 classes (each class represents a digit). In terms of our code, we have `input_dim = 28` and `output_size = 10` for the MNIST dataset.

When we use a batch of input data, the shape of `inputs` is `(batch_size, self.input_dim**2)` and the shape of `labels` is `(batch_size, self.output_size)`, where `batch_size` represents the size of the batch.

The shape of `inputs` is `(batch_size, self.input_dim**2)` because there are `batch_size` images and each image is a square with a width of `input_dim` pixels.

```
batch_size = 16
dataset = dataset.batch(batch_size)
it = dataset.make_one_shot_iterator()
inputs, labels = it.get_next()
with tf.Session() as sess:
    # Batch of data size 16
    input_arr, label_arr = sess.run(
        (inputs, labels))
print(repr(input_arr))
```

Example: A batch of input data and labels. Note that dataset contains all the normalized MNIST data and corresponding labels.

In the example, `inputs` represents the input data tensor, while `labels` represents the one-hot label tensor. The batch size is set to 16 in the example.

Time to Code!

In this section of the course you'll be creating a Python class, `MNISTModel` that represents a CNN for MNIST classification.

Since we aren't using `tf.placeholder` or `tf.Session`, we just need to set the height/width dimension of the image data (`input_dim`) and the number of classes (`output_size`).

Set `self.input_dim` equal to `input_dim` and set `self.output_size` equal to `output_size`.

```
import tensorflow as tf

class MNISTModel(object):
    # Model Initialization
```

```
    def __init__(self, input_dim, output_size):
        # CODE HERE
        pass
```

Reshaping

Reshape image data from the basic 2-D matrix format into NHWC format.

Chapter Goals:

- Learn about NHWC format
- Reshape the input data to NHWC

A. NHWC format

As mentioned in the previous chapter, `inputs` has shape `(batch_size, self.input_dim**2)`. However, in order to use the data with our convolutional neural network, we need to get it into NHWC format.

NHWC format has a shape with four dimensions:

1. Number of image data samples (batch size)
2. Height of each image
3. Width of each image
4. Channels per image

The height and width of each image from the dataset is `self.input_dim`, while the number of channels is 1 (since the images are grayscale). The number of samples, i.e. batch size, is unspecified, since we allow for a variable number of input images.

B. Reshaping the data

The function we use to reshape data in TensorFlow is `tf.reshape`. It takes in a tensor and a new shape as required arguments. The new shape must be able to contain all the elements from the input tensor. For example, we could reshape a tensor from original shape `(5, 4, 2)` to `(4, 10)`, since both shapes contain 40 elements. However, we could not reshape that tensor to `(3, 10, 2)` or `(1, 10)`, since those shapes contain 60 and 10 elements, respectively.

We are allowed to use the special value of -1 in at most one dimension of the

new shape. The dimension with -1 will take on the value necessary to allow the new shape to contain all the elements of the tensor. Using the previous example, if we set a new shape of `(1, 10, -1)`, then the third dimension will have size 4 in order to contain the 40 elements. Since the batch size of our input image data is unspecified, we use -1 for the first dimension when reshaping `inputs`.

```
with tf.Session() as sess:
    input_arr = sess.run(inputs)
    reshaped_arr = sess.run(
        tf.reshape(inputs, [-1, 2, 2, 1])
    )
print(repr(input_arr))
print(repr(reshaped_arr))
```

Example usage of tf.reshape.

In the example, the input tensor, `inputs`, has shape `(batch_size, 4)`. We use `tf.reshape` to convert it to a tensor with shape `(batch_size, 2, 2, 1)`.

Time to Code!

In the next several chapters, we'll be working on completing the `model_layers` function of the `MNISTModel` object. The `model_layers` function sets up the layers of the CNN.

We'll first convert the input data, `inputs`, into NHWC format, with 1 channel. In this case, the H and W dimensions will both be `self.input_dim`, while the C dimension is 1. We use -1 in the N dimension to represent an unspecified batch size.

Set `reshaped_inputs` equal to `tf.reshape` with first argument `inputs` and second argument a list/tuple of four integers representing the NHWC shape.

```
import tensorflow as tf

class MNISTModel(object):
    # Model Initialization
    def __init__(self, input_dim, output_size):
        self.input_dim = input_dim
        self.output_size = output_size
```

```
    self.output_size = output_size

    # CNN Layers
    def model_layers(self, inputs, is_training):
        # CODE HERE
        pass
```

Convolution

Learn how convolutions work and the role they play in CNNs.

Chapter Goals:

- Learn about convolutions
- Write a convolution layer for the neural network

A. Filters and kernels

As mentioned at the end of the **Image Processing** section, filters play a huge role in image recognition. We use filters to transform inputs and extract features that allow our model to recognize certain images. A very high-level example of this would be a curve detecting filter, which allows our model to distinguish between digits with curves (e.g. 8) and digits without curves (e.g. 1).

The weights of a filter are defined through a *kernel matrix*. The kernel is usually a square matrix and its weights are just floating point numbers.

$$\begin{bmatrix} 0.0 & 1.0 & 0.4 \\ 0.1 & 0.0 & 0.0 \\ 1.2 & 0.0 & 1.1 \end{bmatrix}$$

Example kernel weight matrix, with 3x3 dimension.

Like all neural network weights, the filter's weights are trainable variables. We train our neural network (via the kernel matrix weights) to produce filters that are able to extract the most useful hidden features.

When the input data has multiple channels, a filter will have a separate kernel matrix per channel. The MNIST dataset only has one channel, but for other types of image data (e.g. RGB), we would train the model to obtain optimal weights for each channel's kernel matrix.

B. Convolution

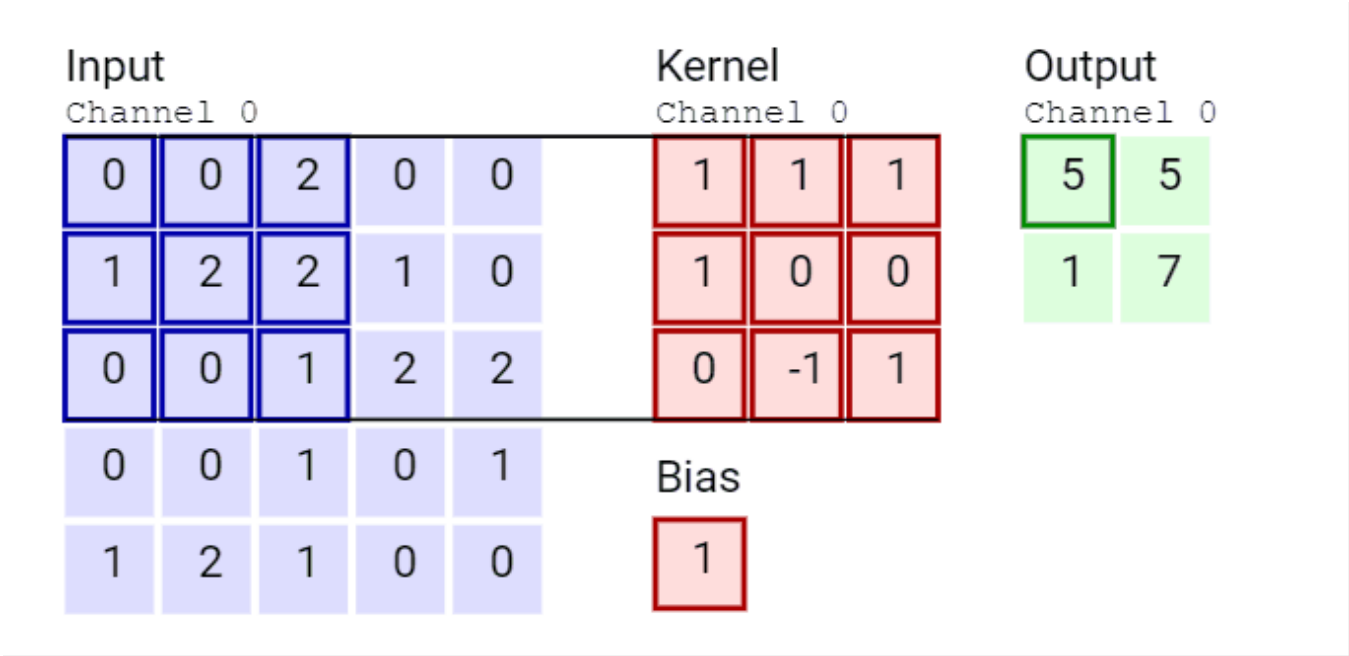
We've now reached the focal point of convolutional neural networks: the convolution. The convolution represents how we apply our filter weights to the input data. The main operation used by a convolution is the matrix dot product, i.e. a summation over the element-wise product of two matrices.

$$\begin{bmatrix} 0.4 & 0.1 \\ 0.1 & 0.2 \end{bmatrix} \cdot \begin{bmatrix} 0.0 & 1.0 \\ 0.4 & 0.2 \end{bmatrix} = 0.18$$

A matrix dot product, where · represents the dot product operation.

In addition to matrix dot products, a convolution also uses a trainable bias term. The bias term is added to the output of each matrix dot product in a convolution.

The number of matrix dot products in a convolution depends on the dimensions of the input data and kernel matrix, as well as the *stride size*. The stride size is the vertical/horizontal offset of the kernel matrix as it moves along the input data. Below is an example of a convolution with a stride size of 2:



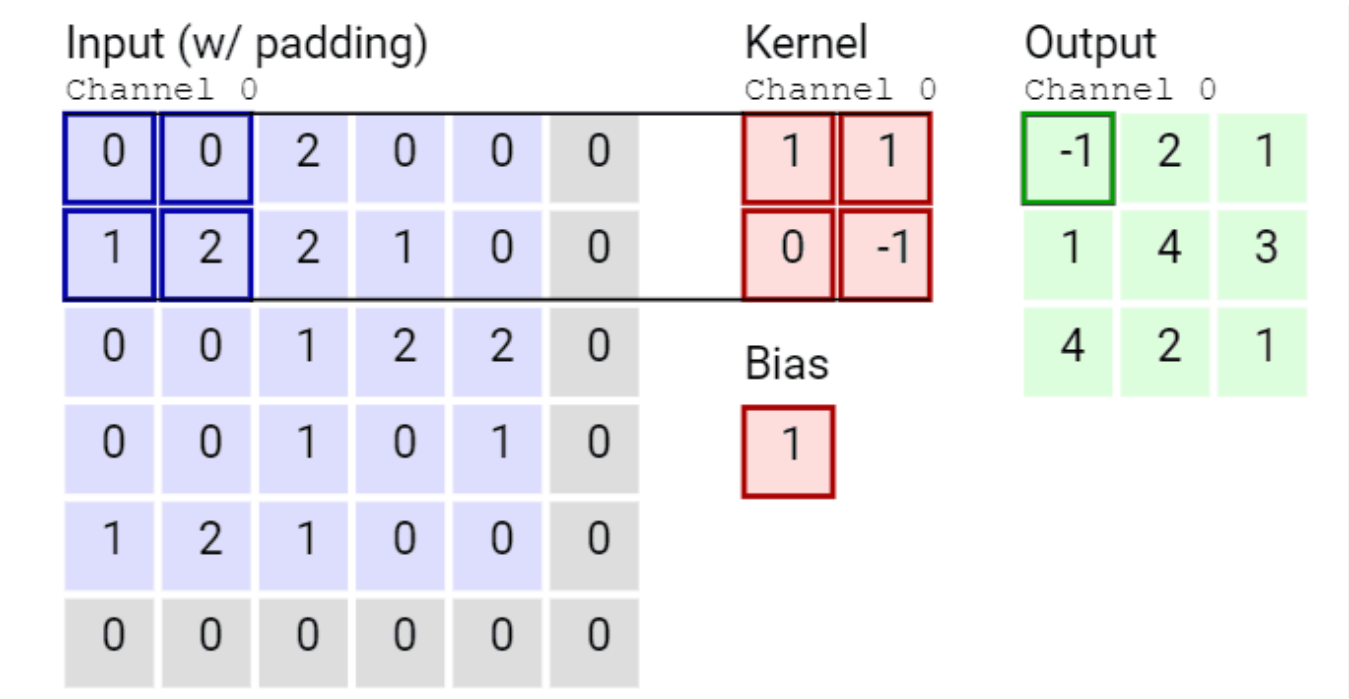
Convolution with stride size of 2 and kernel dimension of 3x3.

C. Padding

The convolution example from the previous section involved a kernel and stride size that fit nicely with the input data. However, if we had instead used a 2x2 kernel, the last row and column of the input data matrix would not have been used in the convolution:

Convolution with stride size of 2 and kernel dimension of 2x2. Since the input dimension is 5x5, the last row and column of the matrix are not used.

If we want to use all the input data in our convolution, we can *pad* the input data matrix with 0's. This means we add rows/columns made entirely of 0's to the edges of the input data matrix. Since 0 multiplied by any number results in 0, the padding doesn't affect matrix dot products. This is important because we don't want to add any distortions to our convolution.



Convolution with stride size of 2 and kernel dimension of 2x2. The input data is padded with 0's on the right and bottom of the matrix.

You might ask why we only padded 0's to the right and bottom of the input

data matrix, rather than around all the edges. To avoid superfluous dot

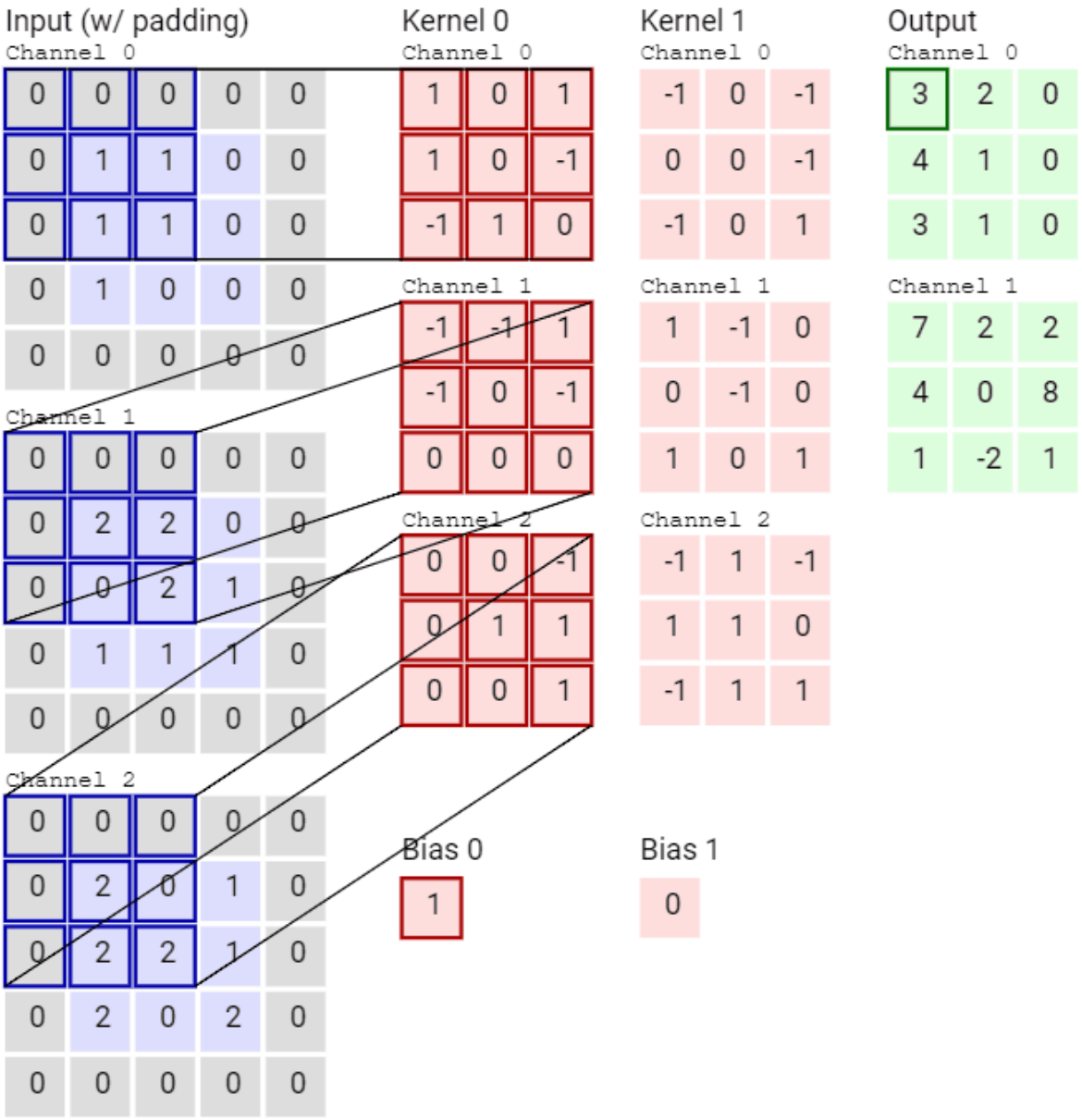
products, we only pad the absolute minimum amount necessary to use all our input data in the convolution.

The padding is distributed as evenly as possible along the top, bottom, left, and right of the matrix. When we have an odd number of padded rows/columns (as in the above example), the additional row/column goes on the bottom and right of the matrix, respectively.

D. Convolution layer

A convolution layer in a CNN applies multiple filters to the input tensor. While each filter has a separate kernel matrix for each of the input channels, the overall result of a filter's convolution is the sum of the convolutions across all the input channels.

Adding more filters to a convolution layer allows the layer to better extract hidden features. However, this comes at the cost of additional training time and computational complexity, since filters add extra weights to the model. The number of channels for the output data is equal to the number of filters the convolution layer uses.



Example convolution layer with 2 filters. The input data has 3 channels and is padded with 0's.

Time to Code!

We'll apply a convolution layer to our `reshaped_inputs` from the previous chapter. In TensorFlow, a convolution layer is implemented with the `tf.layers.conv2d` function. The function takes in the following required arguments:

- `inputs`: The input data tensor.
- `filters`: The number of filters to use.
- `kernel_size`: The height and width dimensions of the kernel matrix.

Some important keyword arguments are:

- `strides`: The stride size for the kernel matrix. Can be a single integer

(same stride size for vertical/horizontal) or a tuple of 2 integers (manually specified vertical/horizontal stride sizes). The first element of the tuple is the vertical stride, the second is the horizontal stride. It defaults to `(1, 1)`.

- `padding`: Either `'valid'` (no padding) or `'same'` (padding).
- `activation`: The activation function to use. Defaults to `None`.
- `name`: The name for the convolution layer (useful for debugging and visualization).

Set `conv1` equal to `tf.layers.conv2d` applied with `reshaped_inputs` as the inputs. The function will use `32` filters, a kernel size of `[5, 5]`, `'same'` padding, and `tf.nn.relu` activation. We'll also set the `name` argument to `'conv1'`.

```
import tensorflow as tf

class MNISTModel(object):
    # Model Initialization
    def __init__(self, input_dim, output_size):
        self.input_dim = input_dim
        self.output_size = output_size

    # CNN Layers
    def model_layers(self, inputs, is_training):
        reshaped_inputs = tf.reshape(
            inputs, [-1, self.input_dim, self.input_dim, 1])
        # CODE HERE
```

Max Pooling

Understand how padding layers can improve a CNN's performance.

Chapter Goals:

- Learn about max pooling and its purpose in CNNs
- Apply max pooling to the output of the convolution layer

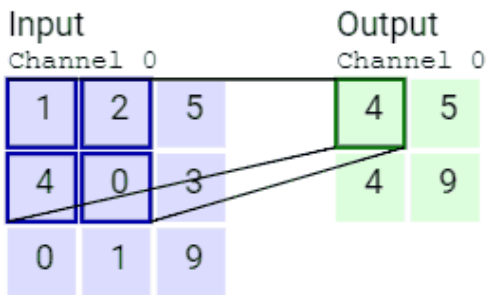
A. Purpose of pooling

While the convolution layer extracts important hidden features, the number of features can still be pretty large. We can use *pooling* to reduce the size of the data in the height and width dimensions. This allows the model to perform fewer computations and ultimately train faster. It also prevents overfitting, by extracting only the most salient features and ignoring potential distortions or uncommon features found in only a few examples.

B. How pooling works

Similar to a convolution, we use filter matrices in pooling. However, the pooling filter doesn't have any weights, nor does it perform matrix dot products. Instead, it applies a reduction operation to subsections of the input data.

The type of pooling that is usually used in CNNs is referred to as *max pooling*. The filters of max pooling use the *max* operation to obtain the maximum number in each submatrix of the input data. An example of max pooling is shown below:



Max pooling with a 2x2 filter and stride size of 1. Note that the output data has reduced dimensions compared to the input data.

Other types of pooling include min pooling and average pooling, which use the *min* and *average* operations, respectively, over subsections of the input data.

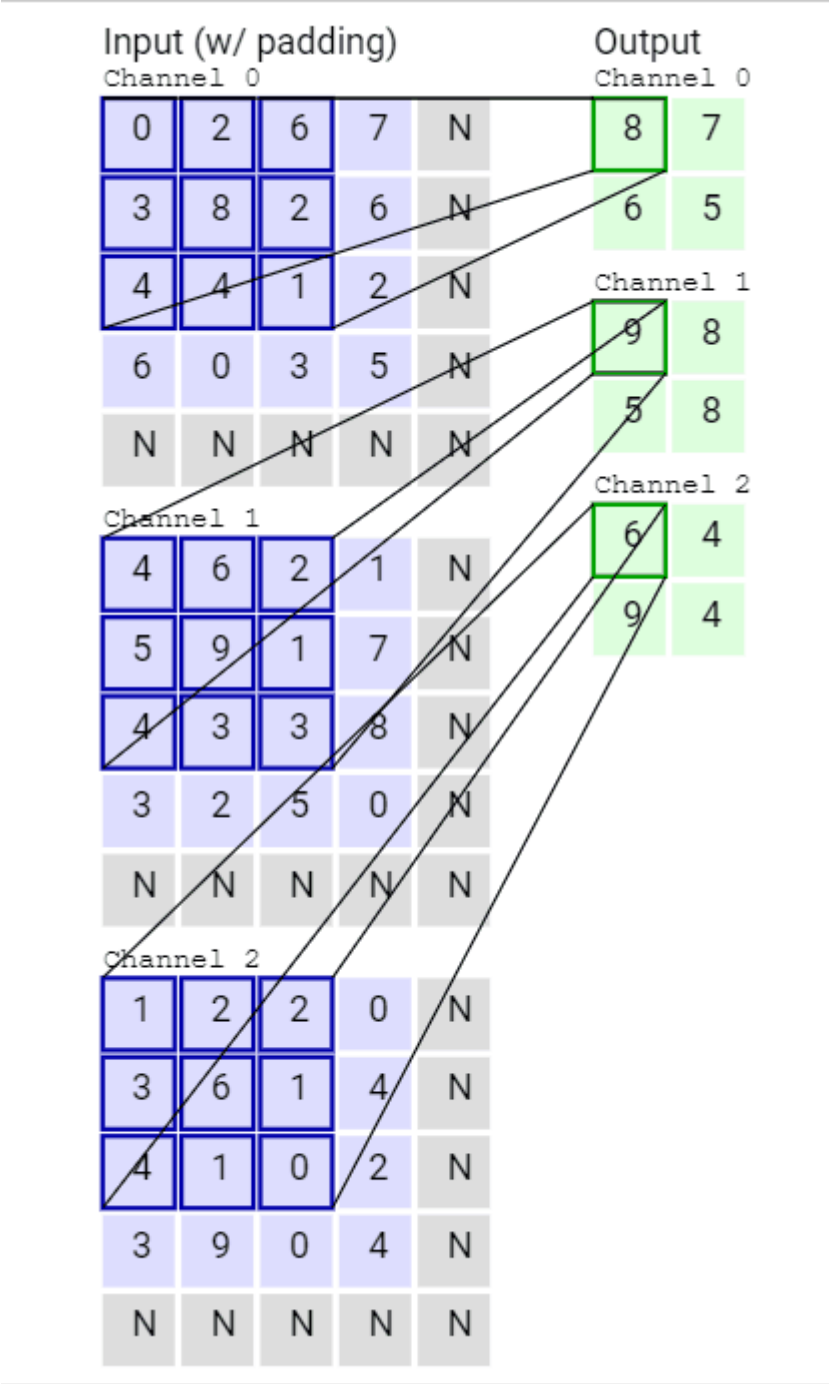
For input data with dimensions $H_{in} \times W_{in}$ the output of pooling with filter dimensions $H_F \times W_F$ and stride size S has the following height and width:

$$H_{out} = \left\lceil \frac{H_{in} - H_F + 1}{S} \right\rceil$$

$$W_{out} = \left\lceil \frac{W_{in} - W_F + 1}{S} \right\rceil$$

C. Padding

Similar to convolutions, we may want to pad our input data prior to pooling. We pad our data with a value dependent on the pooling operation. For example, in max pooling we pad each matrix with -∞. Since -∞ is smaller than every number, it allows us to resize the input data without adding any distortions when pooling. An example of max pooling with padding is shown below:



Max pooling with padding for input data with dimension 4x4. The "N" character represents -∞.

When we use padding, the output dimensions do not depend on the filter dimensions anymore. Specifically, for input data with dimensions $H_{in} \times W_{in}$, the output of padded pooling with a stride size of S has dimensions

$$H_{out} = \left\lceil \frac{H_{in}}{S} \right\rceil$$

$$W_{out} = \left\lceil \frac{W_{in}}{S} \right\rceil$$

Time to Code!

We'll apply max pooling to the output of our first convolution layer from the previous chapter. In TensorFlow, a max pooling layer is implemented with the `tf.layers.max_pooling2d` function. The function takes in the following required arguments:

- `inputs`: The input data tensor.
- `pool_size`: The height and width dimensions of the pooling filter.
- `strides`: The stride size for the kernel matrix. Can be a single integer (same stride size for vertical/horizontal) or a tuple of 2 integers (manually specified vertical/horizontal stride sizes). The first element of the tuple is the vertical stride, the second is the horizontal stride.

The `tf.layers.max_pooling2d` function also takes in a keyword argument for padding. The values are either `'valid'` or `'same'`, with the default value being `'valid'`.

Set `pool1` equal to `tf.layers.max_pooling2d` applied with `conv1` as the inputs. The function will use a pooling size of `[2, 2]` and a stride size of 2, both horizontally and vertically. We'll also set the `name` argument to `'pool1'`.

```
import tensorflow as tf

class MNISTModel(object):
    # Model Initialization
    def __init__(self, input_dim, output_size):
        self.input_dim = input_dim
        self.output_size = output_size

    # CNN Layers
    def model_layers(self, inputs, is_training):
        reshaped_inputs = tf.reshape(
            inputs, [-1, self.input_dim, self.input_dim, 1])
        # Convolutional Layer #1
        conv1 = tf.layers.conv2d(
            inputs=reshaped_inputs,
            filters=32,
            kernel_size=[5, 5],
            padding='same',
            activation=tf.nn.relu,
            name='conv1')
        # CODE HERE
```

Multiple Layers

Discover the nuances of increasing the size and depth of a CNN.

Chapter Goals:

- Add an additional convolution and max pooling layer to the CNN

A. Adding extra layers

Like all neural networks, CNNs can benefit from additional layers. The additional layers allow a CNN to essentially stack multiple filters together for use on the image data. However, similar to building any neural network, we need to be careful of how many additional layers we add. If we add too many layers to a model, we run the risk of having it overfit to the training data and therefore generalizing very poorly. Furthermore, each additional layer adds computational complexity and increases training time for our model.

Since the MNIST images are pretty simple and only have one channel, we won't need more than one additional convolution and max pooling layer. However, in the next two sections of this course you will build CNNs for classifying much more complex images, necessitating the use of many more convolution layers.

B. Increased filters

We usually increase the number of filters in a convolution layer the deeper it is in our model. In this case, our second convolution layer has 64 filters, compared to the 32 filters of the first convolution layer. The deeper the convolution layer, the more detailed the extracted features become. For example, the first convolution layer may have filters that extract features such as lines, edges, and curves. When we get to the second level, the filters of the convolution layer could now extract more distinguishing features, such as the sharp angle of a 7 or the intersecting curves of an 8.

Time to Code!

We're going to add another convolution layer to the model, which will take in the output from the previous pooling layer.

Set `conv2` equal to `tf.layers.conv2d` applied with `pool1` as the inputs. The function will use `64` filters, a kernel size of `[5, 5]`, `'same'` padding, and `tf.nn.relu` activation. We'll also set the `name` argument to `'conv2'`.

We apply max pooling to the output of the added convolution layer.

Set `pool2` equal to `tf.layers.max_pooling2d` applied with `conv2` as the inputs. The function will use a pooling size of `[2, 2]` and a stride size of 2, both horizontally and vertically. We'll also set the `name` argument to `'pool2'`.

```
import tensorflow as tf

class MNISTModel(object):
    # Model Initialization
    def __init__(self, input_dim, output_size):
        self.input_dim = input_dim
        self.output_size = output_size

    # CNN Layers
    def model_layers(self, inputs, is_training):
        reshaped_inputs = tf.reshape(
            inputs, [-1, self.input_dim, self.input_dim, 1])
        # Convolutional Layer #1
        conv1 = tf.layers.conv2d(
            inputs=reshaped_inputs,
            filters=32,
            kernel_size=[5, 5],
            padding='same',
            activation=tf.nn.relu,
            name='conv1')
        # Pooling Layer #1
        pool1 = tf.layers.max_pooling2d(
            inputs=conv1,
            pool_size=[2, 2],
            strides=2,
            name='pool1')
        # CODE HERE
```

Fully-Connected

Understand how fully-connected layers can be used to aggregate and flatten data.

Chapter Goals:

- Convert the NHWC format data into a batch of flattened vectors
- Apply a fully-connected layer to the flattened data

A. Fully-connected layer

We apply a fully-connected layer of size 1024 (i.e. the number of neurons in the layer) to the output data of the second pooling layer. The number of units is somewhat arbitrary. Enough to be powerful, but not so much as to be too resource intensive. The purpose of the fully-connected layer is to aggregate the data features before we convert them to logits. This allows the model to make better predictions than if we had just converted the pooling output directly to logits. We'll use those logits in a later chapter

B. Flattening

The data we have been using in our model is of the NHWC format. However, in order to use a fully-connected layer, we need the data to be a matrix, where the number of rows represents the batch size and the columns represent the data features. Similar to how we reshaped our data in the Reshaping chapter, we'll use `tf.reshape` again. This time, though, we'll be reshaping in the opposite direction and converting *from* NHWC to a 2-D matrix.

Since the first dimension remains the batch size, we'll still use -1 when reshaping. To calculate the size of the second dimension (i.e. the total number of data features in `pool2`), we'll use the inherent `shape` property of tensors in TensorFlow. The `shape` property gives us a `tf.TensorShape` object, which we can convert to a list of integers using its `as_list` function.

```
print(repr(data.shape))
print(repr(data.shape.as_list()))
```

Obtaining the shape of a batch data tensor. Note that "None" represents the (unknown) batch size.

The resulting list of integers contains the NHWC sizes of the tensor. The flattened data size is then just the product of the H, W, and C sizes.

Time to Code!

In this chapter, we'll create a helper function for the `model_layers` function. The helper, `create_fc`, applies a fully-connected layer to the final max-pooling layer of the model.

We first need to get the height, width, and number of channels in `pool2` before we flatten it.

Set `hwc` equal to `pool2.shape.as_list()`, sliced from index `1` to the end.

Using the list of HWC sizes, we can get the total flattened size for the `pool2` data features.

Set `flattened_size` equal to the product of the integers in `hwc`.

We'll flatten the `pool2` tensor using `tf.reshape`. The new shape will have -1 in the first dimension, representing the batch size, and `flattened_size` in the second dimension.

Set `pool2_flat` equal to `tf.reshape` with `pool2` as the first argument and the new shape as the second argument.

We'll now apply a fully-connected layer to the flattened data.

Set `dense` equal to `tf.layers.dense` applied with `pool2_flat` as the inputs, `1024` as the output size, `tf.nn.relu` as the `activation`, and `name` equal to `'dense'`.
Then return `dense`.

```
import tensorflow as tf

class MNISTModel(object):
    # Model Initialization
    def __init__(self, input_dim, output_size):
        self.input_dim = input_dim
```

```
        self.output_size = output_size

    # Apply fully-connected layer
    def create_fc(self, pool2):
        # CODE HERE
        pass

    # CNN Layers
    def model_layers(self, inputs, is_training):
        reshaped_inputs = tf.reshape(
            inputs, [-1, self.input_dim, self.input_dim, 1])
        # Convolutional Layer #1
        conv1 = tf.layers.conv2d(
            inputs=reshaped_inputs,
            filters=32,
            kernel_size=[5, 5],
            padding='same',
            activation=tf.nn.relu,
            name='conv1')
        # Pooling Layer #1
        pool1 = tf.layers.max_pooling2d(
            inputs=conv1,
            pool_size=[2, 2],
            strides=2,
            name='pool1')
        # Convolutional Layer #2
        conv2 = tf.layers.conv2d(
            inputs=pool1,
            filters=64,
            kernel_size=[5, 5],
            padding='same',
            activation=tf.nn.relu,
            name='conv2')
        # Pooling Layer #2
        pool2 = tf.layers.max_pooling2d(
            inputs=conv2,
            pool_size=[2, 2],
            strides=2,
            name='pool2')
        dense = self.create_fc(pool2)
```

Dropout

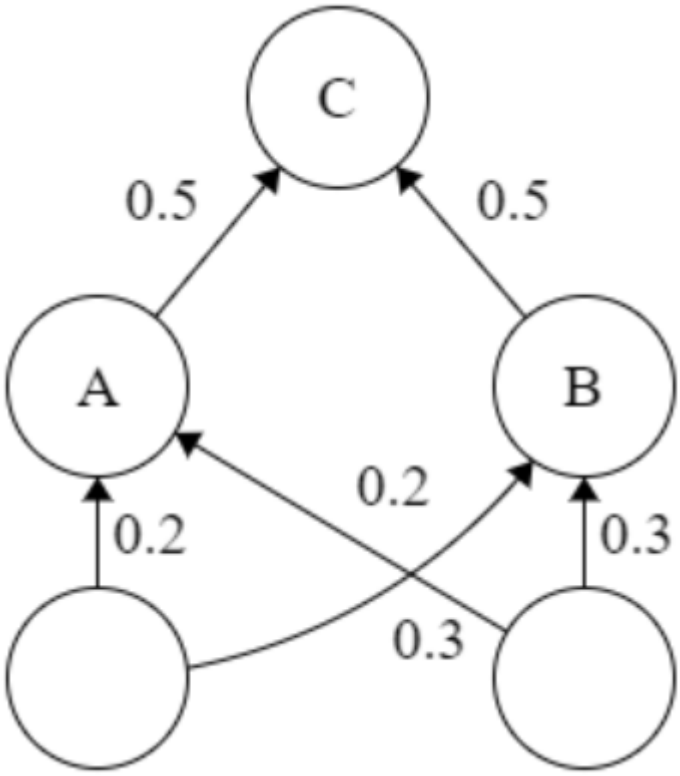
Learn about dropout and how it can reduce overfitting in large neural networks.

Chapter Goals:

- Understand why we use dropout in neural networks
- Apply dropout to a fully-connected layer

A. Co-adaptation

Co-adaptation refers to when multiple neurons in a layer extract the same, or very similar, hidden features from the input data. This can happen when the connection weights for two different neurons are nearly identical.



An example of co-adaptation between neurons A and B. Due to identical weights, A and B will pass the same value into C.

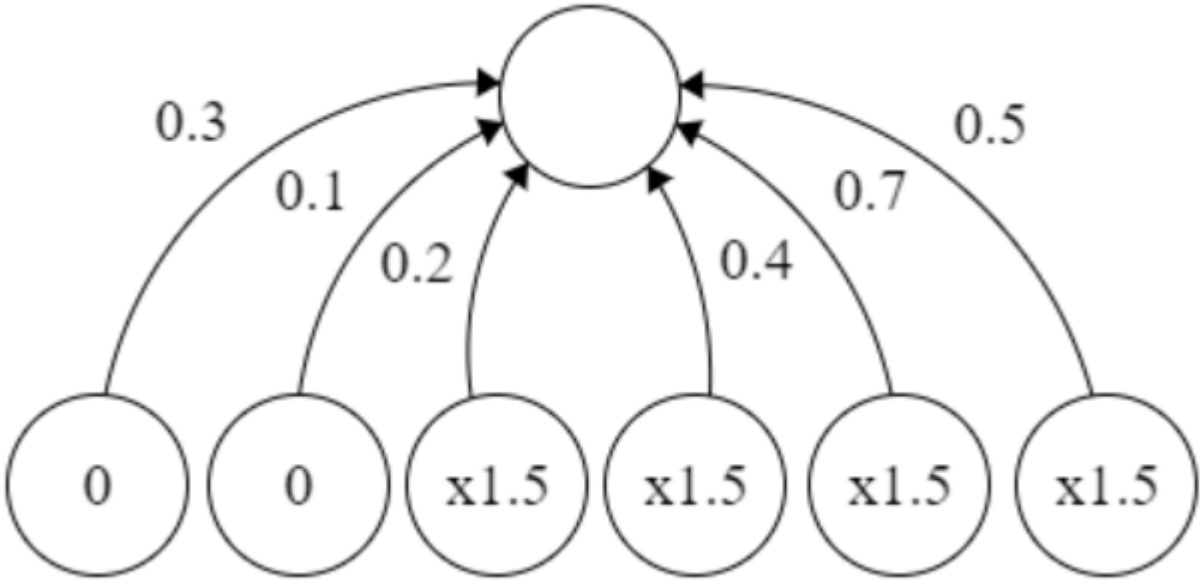
When a fully-connected layer has a large number of neurons, co-adaptation is more likely to occur. This can be a problem for two reasons. First, it is a waste of computation when we have redundant neurons computing the same

output. Second, if many neurons are extracting the same features, it adds

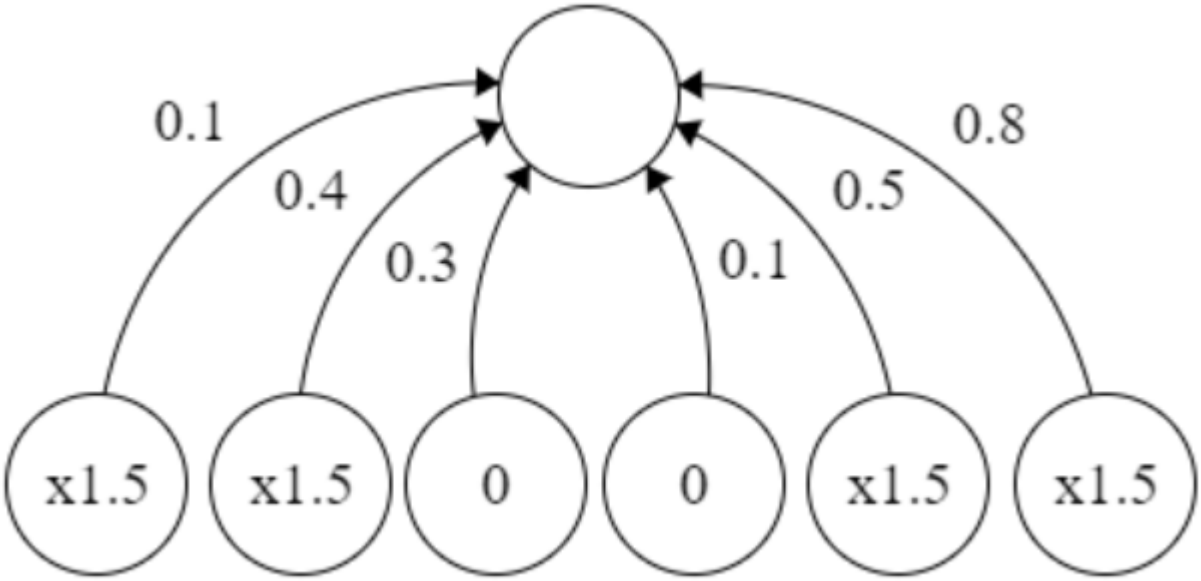
more significance to those features for our model. This leads to overfitting if the duplicate extracted features are specific to only the training set.

B. Dropout

The way we minimize co-adaptation for fully-connected layers with many neurons is by applying *dropout* during training. In dropout, we randomly shut down some fraction of a layer's neurons at each training step by zeroing out the neuron values. The fraction of neurons to be zero'd out is known as the dropout rate, r_d. The remaining neurons have their values multiplied by $\frac{1}{1-r_d}$ so that the overall sum of the neuron values remains the same.



Training Step 1



Training Step 2

Training Step 2

The two images represent dropout applied to a layer of 6 units, shown at multiple training steps. The dropout rate is 1/3, and the remaining 4 neurons at each training step have their value scaled by x1.5.

By randomly dropping a fraction of the neurons, we are essentially choosing a random sample of the neurons to use at each training step. Therefore, each individual neuron works with many different subsets of the other neurons rather than all of them at once. This helps each neuron avoid over-relying on other neurons to correct its mistakes (the underlying cause of co-adaptation), while still allowing the neurons to learn different things from one another.

Time to Code!

In this chapter, we'll create the `apply_dropout` function, which applies dropout to the `dense` fully-connected layer, with a dropout rate of 0.4. In TensorFlow, dropout is implemented via the `tf.layers.dropout` function. The function takes in a required argument of an input tensor, which in this case will be `dense`.

Some important keyword arguments are:

- `rate`: The dropout rate, default is `0.5`.
- `training`: Whether or not the model is in training mode, default is `False`. We only apply dropout during training.

We use the parameter `is_training` to see whether or not we need to apply dropout.

Set `dropout` equal to `tf.layers.dropout` applied with required argument `dense`. We'll set the `rate` argument to `0.4` and `training` argument to `is_training`.
Then return `dropout`.

```
import tensorflow as tf

class MNISTModel(object):
    # Model Initialization
    def __init__(self, input_dim, output_size):
        self.input_dim = input_dim
        self.output_size = output_size
```

```
    # Apply dropout to final layer
    def apply_dropout(self, dense, is_training):

        # CODE HERE
        pass

    # CNN Layers
    def model_layers(self, inputs, is_training):
        reshaped_inputs = tf.reshape(
            inputs, [-1, self.input_dim, self.input_dim, 1])
        # Convolutional Layer #1
        conv1 = tf.layers.conv2d(
            inputs=reshaped_inputs,
            filters=32,
            kernel_size=[5, 5],
            padding='same',
            activation=tf.nn.relu,
            name='conv1')
        # Pooling Layer #1
        pool1 = tf.layers.max_pooling2d(
            inputs=conv1,
            pool_size=[2, 2],
            strides=2,
            name='pool1')
        # Convolutional Layer #2
        conv2 = tf.layers.conv2d(
            inputs=pool1,
            filters=64,
            kernel_size=[5, 5],
            padding='same',
            activation=tf.nn.relu,
            name='conv2')
        # Pooling Layer #2
        pool2 = tf.layers.max_pooling2d(
            inputs=conv2,
            pool_size=[2, 2],
            strides=2,
            name='pool2')
        # Dense Layer
        hwc = pool2.shape.as_list()[1:]
        flattened_size = hwc[0] * hwc[1] * hwc[2]
        pool2_flat = tf.reshape(pool2, [-1, flattened_size])
        dense = tf.layers.dense(pool2_flat, 1024,
            activation=tf.nn.relu, name='dense')
        # Apply Dropout
        dropout = self.apply_dropout(dense, is_training)
```

Logits

Use a fully-connected layer to extract multiclass logits from the CNN.

Chapter Goals:

- Obtain the logits for each digit class

A. Multiclass logits

Since there are 10 possible digits an MNIST image can be, we use a 10 neuron fully-connected layer to obtain the logits for each digit class. The logits are the output of the `model_layers` function.

The rest of the model follows the standard format for multiclass classification:

- Softmax applied to the logits to convert them into per class probabilities
- The `labels` are one-hot vectors, where the "hot index" corresponds to the digit in the MNIST image
- Softmax cross entropy to calculate loss

Time to Code!

In this chapter, we'll create the helper function, `get_logits`, which obtains logits from the previous chapter's `dropout`.

We use a final fully-connected layer to obtain our logits, which we return as the output of our function.

Set `logits` equal to `tf.layers.dense` applied with `dropout` as the inputs, `self.output_size` as the output size, and `name` equal to `'logits'`.
Then return `logits`.

```
import tensorflow as tf

class MNISTModel(object):
    # Model Initialization
    def __init__(self, input_dim, output_size):
        self.input_dim = input_dim
```

```
        self.input_dim = input_dim
        self.output_size = output_size

    # Get logits from the dropout layer
    def get_logits(self, dropout):
        # CODE HERE
        pass

    # CNN Layers
    def model_layers(self, inputs, is_training):
        reshaped_inputs = tf.reshape(
            inputs, [-1, self.input_dim, self.input_dim, 1])
        # Convolutional Layer #1
        conv1 = tf.layers.conv2d(
            inputs=reshaped_inputs,
            filters=32,
            kernel_size=[5, 5],
            padding='same',
            activation=tf.nn.relu,
            name='conv1')
        # Pooling Layer #1
        pool1 = tf.layers.max_pooling2d(
            inputs=conv1,
            pool_size=[2, 2],
            strides=2,
            name='pool1')
        # Convolutional Layer #2
        conv2 = tf.layers.conv2d(
            inputs=pool1,
            filters=64,
            kernel_size=[5, 5],
            padding='same',
            activation=tf.nn.relu,
            name='conv2')
        # Pooling Layer #2
        pool2 = tf.layers.max_pooling2d(
            inputs=conv2,
            pool_size=[2, 2],
            strides=2,
            name='pool2')
        # Dense Layer
        hwc = pool2.shape.as_list()[1:]
        flattened_size = hwc[0] * hwc[1] * hwc[2]
        pool2_flat = tf.reshape(pool2, [-1, flattened_size])
        dense = tf.layers.dense(pool2_flat, 1024,
            activation=tf.nn.relu, name='dense')
        # Apply Dropout
        dropout = tf.layers.dropout(dense, rate=0.4,
            training=is_training)
        # Get and Return Logits
        return self.get_logits(dropout)
```

Classification

Learn how to train and use the CNN model for MNIST datasets.

Chapter Goals:

- Understand how hand-drawn digits are processed and passed into the model for classification

A. Model logistics

The `run_model_setup` function below shows how to set up and train the CNN we've coded:

```
def run_model_setup(self, inputs, labels, is_training):
    logits = self.model_layers(inputs, is_training)

    # convert logits to probabilities with softmax activation
    self.probs = tf.nn.softmax(logits, name='probs')
    # round probabilities
    self.predictions = tf.argmax(
        self.probs, axis=-1, name='predictions')
    class_labels = tf.argmax(labels, axis=-1)
    # find which predictions were correct
    is_correct = tf.equal(
        self.predictions, class_labels)
    is_correct_float = tf.cast(
        is_correct,
        tf.float32)
    # compute ratio of correct to incorrect predictions
    self.accuracy = tf.reduce_mean(
        is_correct_float)
    # train model
    if self.is_training:
        labels_float = tf.cast(
            labels, tf.float32)
        # compute the loss using cross_entropy
        cross_entropy = tf.nn.softmax_cross_entropy_with_logits_v2(
            labels=labels_float,
            logits=logits)
        self.loss = tf.reduce_mean(
            cross_entropy)
        # use adam to train model
        adam = tf.train.AdamOptimizer()
        self.train_op = adam.minimize(
            self.loss, global_step=self.global_step)
```

For more explanation of the code, see the Machine Learning for Software Engineers course on Educative.

B. Real data

After training a model on the MNIST dataset, it is ready to classify real hand-drawn digits. Using the techniques from the **Image Processing** section, we can decode the hand-drawn image to obtain its pixel data (in grayscale format) and then resize it to the same dimensions as the MNIST image data. Since our model inputs have shape `(batch_size, input_dim**2)`, we flatten the image's pixel data and reshape it to `(1, input_dim**2)`.

C. Classifying hand-drawn digits

The code below runs a digit classifier implemented in the backend. It will prompt you to draw a digit. The model will predict which digit you drew.

```
run_digit_recognizer()
```

Quiz

1

Why do we reshape the input data into NHWC format for a CNN?

COMPLETED 0%

1 of 4

Introduction

In this section of the course you will be building a memory-efficient CNN known as SqueezeNet. Although small, its performance is on par with much larger models such as AlexNet, the breakthrough deep CNN that won the 2012 ImageNet Challenge. We will train the model on the CIFAR-10 dataset, which contains 60,000 images in 10 different classes.

A. Memory Usage

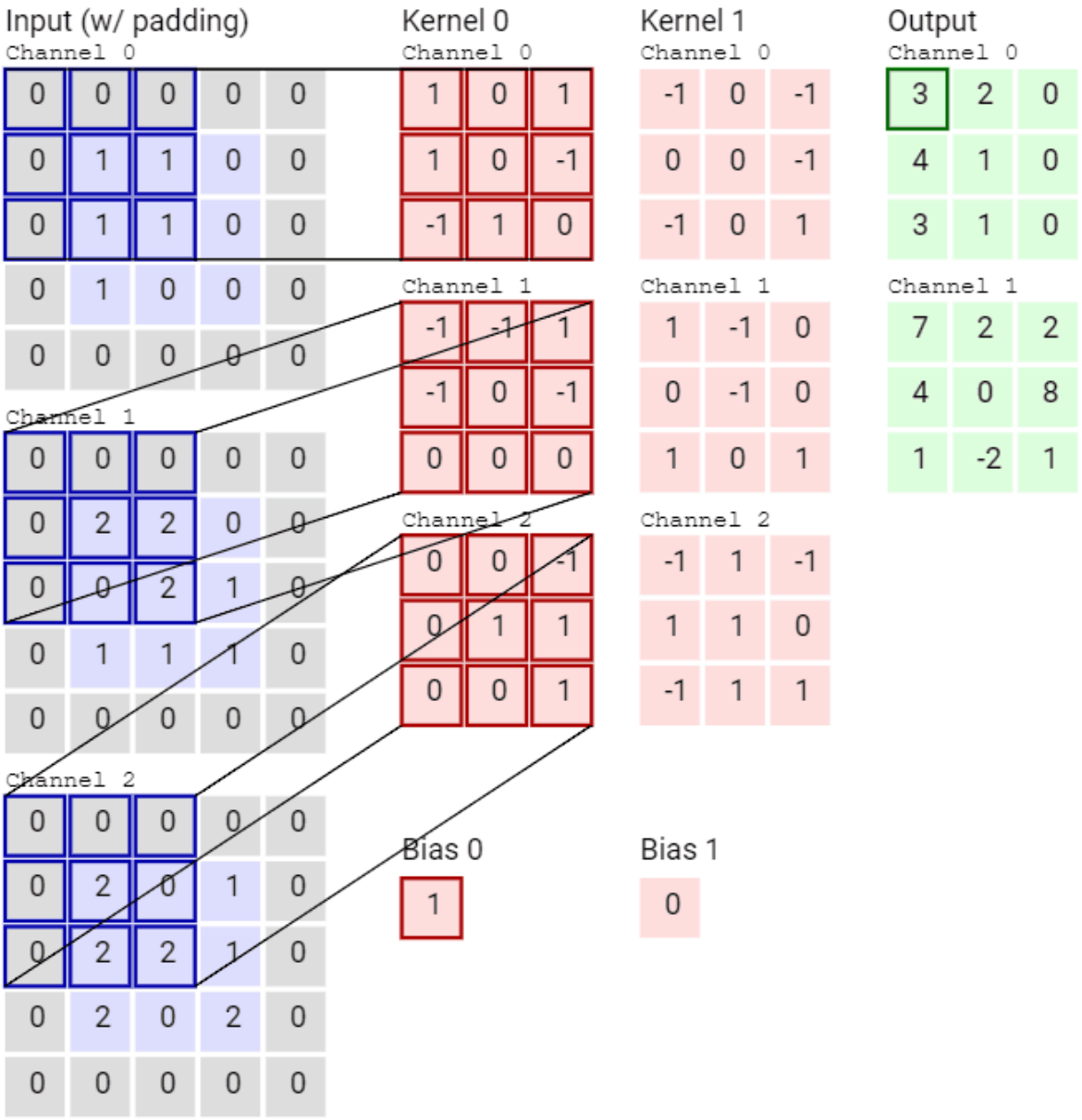
While we're normally concerned with model accuracy, the amount of memory a model uses is important as well. After training a model, we store its computation graph and parameters (weights + biases) for future use. Though a model's computation graph is relatively small (since even large models won't have more than a couple hundred layers), the number of parameters a model has can be in the millions.

Most high performance models require hundreds of MB of space to store their parameters. The aforementioned AlexNet uses over 200MB for storage of 60 million parameters. On the other hand, the SqueezeNet model architecture uses less than 1MB. That's even less memory than the simple digit recognition model we built in the previous chapter (13MB). However, SqueezeNet has significantly better performance than our previous model and actually matches AlexNet in accuracy.

When accuracy between two models is relatively equal, we prefer the model that uses up less memory.

B. Calculating parameters

To understand model sizes, we need to be able to calculate the number of parameters in a model. Let’s take a look at an example convolution layer:



Convolution layer with 2 filters. The input data has 3 channels and each kernel has dimension 3x3.

The convolution layer parameters are the kernel weights and biases. Each kernel has 3x3 dimensions, and there are 3 kernels for each of the 2 filters. Therefore, the total number of kernel weights is $3 \times 3 \times 2 \times 3 = 54$. There are 2 biases (one per filter), so the total number of parameters in this convolution layer is 56. In general, for a convolution layer with C input channels, F filters, and kernels with dimension $H_K \times W_K$, the number of parameters, P, is

$$P = H_K \times W_K \times F \times C + F$$

To calculate the total number of parameters for a model, we just aggregate the number of parameters across each convolution layer.

C. SqueezeNet

There are quite a few benefits of using a smaller model such as SqueezeNet. First, it takes less time to train and load a smaller model. When performance is comparable, it is preferable to use a model that takes less time to train and set up.

Furthermore, when memory usage is limited, it may not even be possible to store a large model. For example, if you're working in an environment with a limited disk quota (such as a Linux container), storing a large model may exceed the memory limit. This problem is especially prevalent when we are storing multiple variations of a large model (which is often the case during training and experimentation). However, this would not be an issue with a model based on the SqueezeNet architecture.

Initialization

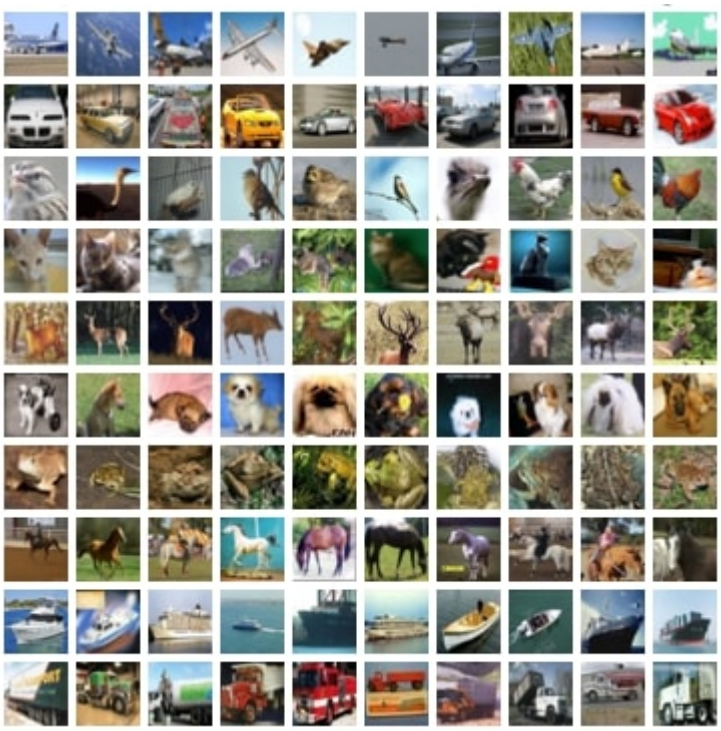
Learn about the CIFAR-10 dataset along with the input and output sizes of the data.

Chapter Goals:

- Learn about the CIFAR-10 dataset
- Initialize the model with data dimensions

A. CIFAR-10

The CIFAR-10 (Canadian Institute for Advanced Research) dataset contains 60,000 color images with dimensions 32x32. The images are distributed evenly across 10 categories: **airplane**, **automobile**, **bird**, **cat**, **deer**, **dog**, **frog**, **horse**, **ship**, **truck**. We split the dataset into 50,000 images for training and 10,000 images for testing.



Example images from the CIFAR-10 dataset.

The CIFAR-10 dataset is available for download (along with the 100 category version, CIFAR-100), on Alex Krizhevsky's website.

(Fun fact: Alex Krizhevsky helped invent the AlexNet model mentioned in the previous chapter

(Fun fact: Alex Krizhevsky helped invent the AlexNet model mentioned in the previous chapter, which incidentally was named after him. The LeNet model from the **CNN** section was also named after its inventor, Yann LeCun. SqueezeNet, however, is not named after its developers.)

B. Inputs and labels

Each image has dimensions 32x32, meaning `original_dim` will be 32 for the CIFAR-10 dataset. The images have color, so they follow the RGB format, meaning each pixel contains three integers (one each for the red, blue, and green channels). In total, an image is represented by

$$32 \times 32 \times 3 = 1024 \times 3 = 3072$$

integers between 0 and 255. The first 1024 integers represent the red channel pixel values, the next 1024 represent the blue channel pixel values, and the final 1024 represent the green channel pixel values.

The labels this time are just single integers corresponding to the class index of the image, rather than one-hot vectors. This is referred to as a *sparse representation* of the labels. However, `output_size` is still equal to the number of image categories, in this case 10.

For a batch of input data, the shape of `inputs` is `(batch_size, 3 * self.original_dim**2)` and the shape of `labels` is `(batch_size,)`, where `batch_size` represents the size of the batch. Due to the sparse representation, `labels` is a 1-D tensor (1-D tensors have a trailing comma in their shape).

```
batch_size = 32
dataset = dataset.batch(batch_size)
it = dataset.make_one_shot_iterator()
inputs, labels = it.get_next()
with tf.Session() as sess:
    # Batch of data size 10
    input_arr, label_arr = sess.run(
        (inputs, labels))
```

Example batch of input data and labels. Note that dataset contains all the CIFAR-10 data and corresponding labels.

In the example, `inputs` represents the input data tensor, while `labels` represents the 1-D label tensor. The batch size is set to 32 in the example.

Time to Code!

In this section of the course you'll be creating a Python class, `SqueezeNetModel` that represents the SqueezeNet model you'll be building.

We'll set the original height/width dimension of the image data (`original_dim`) and the number of classes (`output_size`).

Inside the `__init__` function, set `self.original_dim` equal to `original_dim` and set `self.output_size` equal to `output_size`.

In the next chapter we'll be doing image processing, so we need to set the resized dimension of the images.

Set `self.resize_dim` equal to `resize_dim`.

```
import tensorflow as tf

class SqueezeNetModel(object):
    # Model Initialization
    def __init__(self, original_dim, resize_dim, output_size):
        # CODE HERE
        pass
```

Augmentation

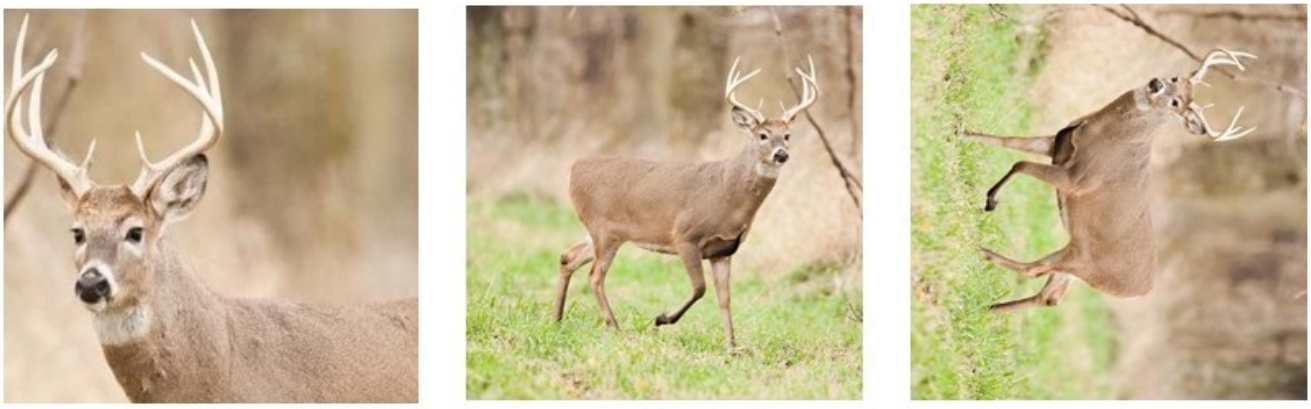
Discover how data augmentation works and its benefits for training a CNN.

Chapter Goals:

- Learn about data augmentation and its purpose
- Apply data augmentation to image data
- Standardize the image data

A. Data augmentation

Data augmentation is the process of artificially enlarging a dataset through *image transformations*. No matter how good a model is, it cannot perform at its fullest potential unless we have a large enough dataset. Although CIFAR-10 contains 50,000 training images, it would benefit our model if it were trained on more data. Rather than going through the process of obtaining new images and resizing them, we can instead perform transformations on the images (e.g. rotating or cropping) to manufacture "new" image data.



The transformations used are (from left to right): crop, horizontal flip, and rotation.

Using data augmentation, we create a variation of each image in a batch of input data. Since we train our model on hundreds of epochs (full run-throughs) of the CIFAR-10 dataset, data augmentation constructs hundreds of variations for each image in the training set.

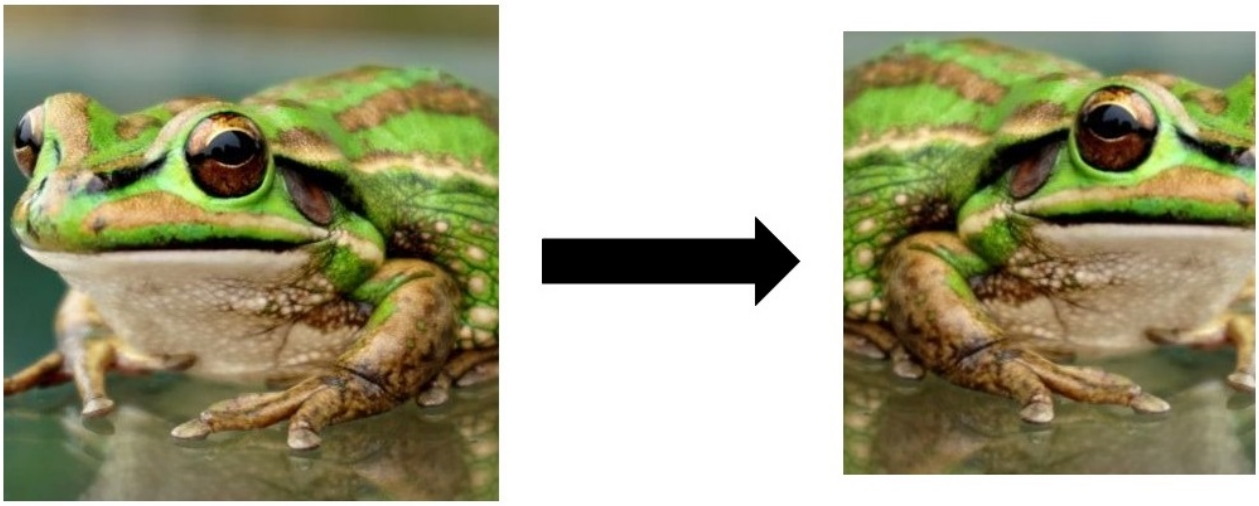
B. Image transformation

In our code, the image transformation we use is a random crop followed (potentially) by a horizontal flip. We don't use rotations because it is unrealistic to see most of the CIFAR-10 images rotated in real life.

We first apply a random crop of the image to a height and width of `resize_dim`. This means that we crop the original image data to a randomly chosen submatrix of the data with height and width equal to `resize_dim`. For the CIFAR-10 dataset, we set `resize_dim = 24`.

Then, with a probability of 0.5, we will flip the cropped image horizontally. This results in approximately half of our augmented data being flipped and

the other half not flipped, which is the optimal distribution for data variety.



Random crop and flip applied to a frog input image.

Since we only apply data augmentation when training our model, we don't use any image transformations when the model is not training. However, because we apply a random crop while training, the input to our model layers must now have height and width equal to `resize_dim`. Therefore, we need to resize the image data even during model evaluation. For consistency between training and evaluation, we choose to use a central crop (submatrix centered in the middle) of the original image to resize the data.

C. Data standardization

The CIFAR-10 image data is still pixel integers between 0-255, so we need to convert the data to a more suitable range of values. Rather than normalize to a range of 0.0 to 1.0 (like the MNIST data), we do per image *standardization*. What this means is that we linearly scale the pixel data of an image so that the new data has zero mean and unit variance. To accomplish this, we use the function `tf.image.per_image_standardization`, which takes in image data and returns the standardized data.

Time to Code!

In this chapter you'll be completing the `image_preprocessing` function, which preprocesses image data to augment the dataset.

Specifically, you'll be creating the helper function `random_crop_and_flip`, for cropping and flipping an image during training.

We'll first apply a random crop and random horizontal flip to the image data. We use `tf.random_crop` for the random crop, with a new cropped shape of `[self.resize_dim, self.resize_dim, 3]`.

Set `crop_image` equal to `tf.random_crop` with first argument `float_image` and second argument the new cropped shape.

We then pass the cropped image as the required argument for `tf.image.random_flip_left_right`, which performs the random flip.

Set `updated_image` equal to `tf.image.random_flip_left_right` applied with required argument `crop_image`.
Then return `updated_image`.

```
import tensorflow as tf

class SqueezeNetModel(object):
    # Model Initialization
    def __init__(self, original_dim, resize_dim, output_size):
        self.original_dim = original_dim
        self.resize_dim = resize_dim
        self.output_size = output_size

    # Random crop and flip
    def random_crop_and_flip(self, float_image):
        # CODE HERE
        pass

    # Data Augmentation
    def image_preprocessing(self, data, is_training):
        reshaped_image = tf.reshape(data, [3, self.original_dim, self.original_dim])
        transposed_image = tf.transpose(reshaped_image, [1, 2, 0])
        float_image = tf.cast(transposed_image, tf.float32)
        if is_training:
            updated_image = self.random_crop_and_flip(float_image)
        else:
            updated_image = tf.image.resize_image_with_crop_or_pad(float_image, self.resize_d
        standardized_image = tf.image.per_image_standardization(updated_image)
        return standardized_image
```

Fire Module

Learn about the central component of SqueezeNet, the fire module.

Chapter Goals:

- Learn strategies for decreasing the number of parameters in a model
- Understand how the fire module works and why it's effective
- Write your own fire module function

A. Decreasing parameters

In order to make a smaller model, we need to decrease the number of weights per convolution layer. There are three ways to decrease the number of weights in a convolution layer:

- Decrease the kernel size
- Decrease the number of filters used
- Decrease the number of input channels

We don't necessarily want to reduce the number of filters we use, since using a variety of filters allows us to extract different hidden features from the input. However, there are ways to decrease the kernel size and number of input channels while still maintaining good model performance.

B. Kernel size

The size of a kernel represents the amount of spatial information it can capture. For example, a 1x1 kernel will only capture the channel information for individual pixels, while a 3x3 kernel will aggregate the information between adjacent pixels within each 3x3 square of the input data.

Although larger kernels can capture more information, it comes at the cost of additional parameters. A convolution layer that uses 3x3 kernels will use 9x as many parameters as a layer that uses 1x1 kernels. A good strategy for balancing performance and parameter count is to use a mix of larger and smaller size kernels.

smaller size kernels.

C. Intermediate layer

The way we decrease the number of input channels is by adding an intermediate convolution layer. Though this may seem counter-intuitive, since adding an extra layer introduces additional kernel weights, it can drastically decrease the number of parameters used in a layer. Consider a convolution layer with 100 filters and 3x3 kernels. Given the input has 50 channels, we can use the equation from chapter 1 to calculate the number of parameters in the layer:

$$P = 3 \times 3 \times 100 \times 50 + 100 = 45{,}100$$

Now, consider the case where we first apply an intermediate convolution layer with 10 filters and 1x1 kernels. The intermediate output will have 10 channels. The number of parameters used in the intermediate layer, P_I, is

$$P_I = 1 \times 1 \times 10 \times 50 + 10 = 510$$

Then if we pass the intermediate output into our original convolution layer, the total number of parameters used becomes

$$P = P_I + 3 \times 3 \times 100 \times 10 + 100 = 9{,}610$$

By adding the intermediate layer we decrease the number of parameters by a factor of nearly 5.

D. Fire module

The fire module, the key building block of SqueezeNet, incorporates the ideas from the previous two sections. It uses an intermediate convolution layer, referred to as a *squeeze layer*, then passes the intermediate output into an *expand layer* with a larger number of filters.

The expand layer contains two convolution layers with an equal number of filters. One of the layers uses 1x1 kernels, while the other uses 3x3 kernels. The mix of 1x1 kernels decreases the number of parameters used. The outputs of the two layers have the same size, since both layers use the same number of filters.

The two outputs are then concatenated along the channel dimension (doubling the number of channels) to produce the overall output of the fire

module.

To concatenate, we use the function `tf.concat`, which takes in two required arguments:

- `values`: A list of tensors to concatenate.
- `axis`: The dimension to concatenate along.

Since the channel dimension is the last dimension, we use `-1` as the `axis`.

```
import tensorflow as tf

print(expand1x1.shape)
print(expand3x3.shape)
output = tf.concat([expand1x1, expand3x3], axis=-1)
print(output.shape)
```

Example: Concatenation of expand1x1 and expand3x3 in the fire module.

The ratio of the number of filters in the squeeze layer vs. the expand layer is known as the *squeeze ratio*. A larger squeeze ratio (i.e. increasing the number of filters in the squeeze layer) can improve the model performance up to a certain extent, at the cost of increased parameter count.

Below is the full code for a fire module:

```
import tensorflow as tf

class SqueezeNetModel(object):
    # __init__ and other functions omitted

    # Convolution layer wrapper
    def custom_conv2d(self, inputs, filters, kernel_size, name):
        return tf.layers.conv2d(
            inputs=inputs,
            filters=filters,
            kernel_size=kernel_size,
            activation=tf.nn.relu,
            padding='same',
            name=name)

    # SqueezeNet fire module
    def fire_module(self, inputs, squeeze_depth, expand_depth, name):
        with tf.variable_scope(name):
            squeezed_inputs = self.custom_conv2d(
                inputs,
                squeeze_depth,
```

```
        [1, 1],
        'squeeze')
    expand1x1 = self.custom_conv2d(
        squeezed_inputs,
        expand_depth,
        [1, 1],
        'expand1x1')
    expand3x3 = self.custom_conv2d(
        squeezed_inputs,
        expand_depth,
        [3, 3],
        'expand3x3')
    return tf.concat([expand1x1, expand3x3], axis=-1)
```

Fire module and conv layer wrapper within the SqueezeNetModel.

Stacking Fire

Stack multiple fire modules together by creating a utility function.

Chapter Goals:

- Write a utility function to stack multiple fire modules

A. Utility function

While the SqueezeNet model uses very few parameters, it still has many layers. The original architecture, which was built for the larger ImageNet dataset, uses 8 fire modules. Our model is a condensed version, and only uses 4. However, that is still several fire modules, making it useful to write a utility function that can stack multiple layers.

In general, when we deal with more complex model architectures, there are going to be repetitions of the main building blocks in the model. Rather than writing layers upon layers of the same code, we can create a utility function that allows us to combine multiple building blocks. This is especially helpful for models that have dozens or even hundreds of layers, which is the case in the **ResNet** section.

Time to Code!

In this chapter, you'll be stacking fire modules by using the `fire_module` function from the previous chapter. The function that you'll complete which performs this task is `multi_fire_module` (line **41**).

The `params_list` argument is a list of tuples, where each tuple represents the arguments for a fire module, i.e. `(squeeze_depth, expand_depth, name)`. We'll loop through the `params_list` to create each of the fire modules.

Create a `for` loop that goes through each `params` tuple in `params_list`.

The `layer` argument is the initial input data for our multi-fire module. We'll use it as the `inputs` argument for each fire module, as well as set it to the

output of each fire module. This way we can continuously reuse the same variable.

Inside the `for` loop, set `layer` equal to `self.fire_module` applied with `layer` as the first argument and the remaining three arguments coming from `params`.

After the end of the `for` loop, `layer` represents the output of the final fire module.

Outside the `for` loop, return `layer`.

```
import tensorflow as tf

class SqueezeNetModel(object):
    # Model Initialization
    def __init__(self, original_dim, resize_dim, output_size):
        self.original_dim = original_dim
        self.resize_dim = resize_dim
        self.output_size = output_size

    # Convolution layer wrapper
    def custom_conv2d(self, inputs, filters, kernel_size, name):
        return tf.layers.conv2d(
            inputs=inputs,
            filters=filters,
            kernel_size=kernel_size,
            activation=tf.nn.relu,
            padding='same',
            name=name)

    # SqueezeNet fire module
    def fire_module(self, inputs, squeeze_depth, expand_depth, name):
        with tf.variable_scope(name):
            squeezed_inputs = self.custom_conv2d(
                inputs,
                squeeze_depth,
                [1, 1],
                'squeeze')
            expand1x1 = self.custom_conv2d(
                squeezed_inputs,
                expand_depth,
                [1, 1],
                'expand1x1')
            expand3x3 = self.custom_conv2d(
                squeezed_inputs,
                expand_depth,
                [3, 3],
                'expand3x3')
            return tf.concat([expand1x1, expand3x3], axis=-1)

    # Stacked fire modules
    def multi_fire_module(self, layer, params_list):
        # CODE HERE
        pass
```

Model Layers

Learn about the high-level architecture of SqueezeNet.

Chapter Goals:

- Learn the high-level architecture of SqueezeNet
- Understand the use of non-fire module convolution layers

A. Overview

As mentioned in the previous chapter, we'll be building a condensed version of the SqueezeNet model. The differences between our model and the original are:

- Our model uses only the first 4 fire modules from the original SqueezeNet (rather than all 8)
- The initial convolution layer for our model uses fewer filters (64 vs. 96) and a smaller kernel size (3x3 vs. 7x7)
- The max pooling layers in our model use smaller filters (2x2 vs. 3x3)

The changes are mainly due to the fact that our dataset is CIFAR-10, which has significantly smaller images and fewer categories than ImageNet, the dataset used to train the original SqueezeNet. We go in-depth on the ImageNet dataset in the **ResNet** section.

B. Initial layers

We start off with a regular convolution layer, rather than a fire module, to apply the initial filters to our image data. This is because we don't want to start off with squeezing our image data and potentially losing out on important general features. It's usually a good idea for any CNN to start off with a regular convolution layer.

Likewise, it's a good idea to apply max pooling after the initial convolution layer. Although it's less of a need with small images like in the CIFAR-10 dataset, reducing the dimension of the initial data helps our model train faster

dataset, reducing the dimension of the initial data helps our model train faster and pick up on important features. In our code, each of the max pooling layers

has a stride size of 2 and a filter size of 2x2, so the data's width and height are halved after each max pooling layer.

Time to Code!

In this chapter you'll be building the first convolution+pooling layer in the `model_layers` function (line **30**).

We start our model off with a regular convolution layer.

Set `conv1` equal to `self.custom_conv2d` applied with `inputs` as the first argument, `64` as the number of `filters`, `[3, 3]` as the `kernel_size`, and `'conv1'` as the `name`.

Next we apply max pooling to the convolution layer's output. We provide a wrapper function for max pooling called `custom_max_pooling2d`. This automatically sets the `strides` argument to `2` and the `pool_size` (i.e. filter size) to `[2, 2]`.

Set `pool1` equal to `self.custom_max_pooling2d` applied with `conv1` as the first argument and `'pool1'` as the `name`.

```
import tensorflow as tf

class SqueezeNetModel(object):
    # Model Initialization
    def __init__(self, original_dim, resize_dim, output_size):
        self.original_dim = original_dim
        self.resize_dim = resize_dim
        self.output_size = output_size

    # Convolution layer wrapper
    def custom_conv2d(self, inputs, filters, kernel_size, name):
        return tf.layers.conv2d(
            inputs=inputs,
            filters=filters,
            kernel_size=kernel_size,
            activation=tf.nn.relu,
            padding='same',
            name=name)

    # Max pooling layer wrapper
    def custom_max_pooling2d(self, inputs, name):
        return tf.layers.max_pooling2d(
            inputs=inputs,
            pool_size=[2, 2],
            strides=2,
```

```
            name=name)

    # Model Layers

    # inputs: [batch_size, resize_dim, resize_dim, 3]
    def model_layers(self, inputs, is_training):
        # CODE HERE
        pass
```

Adding Fire

Add fire modules to the model and learn about delayed downsampling.

Chapter Goals:

- Add a multi-fire module block to the model
- Learn about delayed downsampling

A. Delayed downsampling

While reducing the data dimensions via max pooling can help our model training, strategic placement of max pooling layers can improve the SqueezeNet model's accuracy. Rather than applying max pooling within the first fire module (e.g. after the squeeze layer), we instead apply it after the first two fire modules.

Placing the max pooling layers later in the model structure is referred to as *delayed downsampling*. We wait until later in the model structure to *downsample* the data (i.e. reduce height/width dimensions through pooling), so the earlier convolution layers can have a larger dimension input. The creators of SqueezeNet showed that having larger dimension inputs for more layers in the model can help improve its accuracy.

Time to Code!

In this chapter, we'll continue building the `model_layers` function by adding the first multi-fire module.

We'll start by setting the list of parameter tuples to describe our fire modules. The list will contain two tuples representing the `fire_module` arguments. For both fire modules, we use `32` as the squeeze depth and `64` as the expand depth. The first fire module will be named `'fire1'` while the second will be named `'fire2'`.

Set `fire_params1` equal to a list of two tuples with the specified values.

Now we apply the multi-fire module block with our list of tuple parameters.

Set `multi_fire1` equal to `self.multi_fire_module` with `pool1` as the first argument and `fire_params1` as the second argument.

We perform max pooling on the output of the multi-fire module block.

Set `pool2` equal to `self.custom_max_pooling2d` applied with `multi_fire1` as the first argument and `'pool2'` as the `name`.

```
import tensorflow as tf

class SqueezeNetModel(object):
    # Model Initialization
    def __init__(self, original_dim, resize_dim, output_size):
        self.original_dim = original_dim
        self.resize_dim = resize_dim
        self.output_size = output_size

    # Convolution layer wrapper
    def custom_conv2d(self, inputs, filters, kernel_size, name):
        return tf.layers.conv2d(
            inputs=inputs,
            filters=filters,
            kernel_size=kernel_size,
            activation=tf.nn.relu,
            padding='same',
            name=name)

    # Max pooling layer wrapper
    def custom_max_pooling2d(self, inputs, name):
        return tf.layers.max_pooling2d(
            inputs=inputs,
            pool_size=[2, 2],
            strides=2,
            name=name)

    # SqueezeNet fire module
    def fire_module(self, inputs, squeeze_depth, expand_depth, name):
        with tf.variable_scope(name):
            squeezed_inputs = self.custom_conv2d(
                inputs,
                squeeze_depth,
                [1, 1],
                'squeeze')
            expand1x1 = self.custom_conv2d(
                squeezed_inputs,
                expand_depth,
                [1, 1],
                'expand1x1')
            expand3x3 = self.custom_conv2d(
                squeezed_inputs,
                expand_depth,
                [3, 3],
                'expand3x3')
            return tf.concat([expand1x1, expand3x3], axis=-1)
```

```
    # Utility function for multiple fire modules
    def multi_fire_module(self, layer, params_list):

        for params in params_list:
            layer = self.fire_module(
                layer,
                params[0],
                params[1],
                params[2])
        return layer

    # Model Layers
    # inputs: [batch_size, resize_dim, resize_dim, 3]
    def model_layers(self, inputs, is_training):
        conv1 = self.custom_conv2d(
            inputs,
            64,
            [3, 3],
            'conv1')
        pool1 = self.custom_max_pooling2d(
            conv1,
            'pool1')
        # CODE HERE
```

Depth

Increase the SqueezeNet model's depth for added performance.

Chapter Goals:

- Add depth to the model through additional fire modules

A. Deeper fire modules

In order to extract more distinguishing features from the data, we add more fire modules to our model. The additional fire modules will use twice as many filters in the expand layer (from 64 to 128). This follows the same approach outlined in the **CNN** section, where we increase the number of filters used in the convolution layers deeper into the model.

To avoid overfitting, we apply dropout after our second multi-fire module block, with a dropout rate of 0.5. There is no max pooling layer after this multi-fire module block, since we'll obtain logits in the next chapter using average pooling.

The code below shows the `model_layers` function with added depth:

```
import tensorflow as tf

class SqueezeNetModel(object):
    # __init__ and other functions omitted

    # Model Layers
    def model_layers(self, inputs, is_training):
        conv1 = self.custom_conv2d(
            inputs,
            64,
            [3, 3],
            'conv1')
        pool1 = self.custom_max_pooling2d(
            conv1,
            'pool1')
        fire_params1 = [
            (32, 64, 'fire1'),
            (32, 64, 'fire2')
        ]
        multi_fire1 = self.multi_fire_module(
            pool1,
```

```
        pool1,
        fire_params1)
    pool2 = self.custom_max_pooling2d(
        multi_fire1,
        'pool2')
    fire_params2 = [
        (32, 128, 'fire3'),
        (32, 128, 'fire4')
    ]
    multi_fire2 = self.multi_fire_module(
        pool2,
        fire_params2)
    dropout1 = tf.layers.dropout(multi_fire2, rate=0.5,
        training=is_training)
```

Logits

Use global average pooling to obtain the model's logits.

Chapter Goals:

- Learn about global average pooling and its benefits
- Obtain the per class model logits

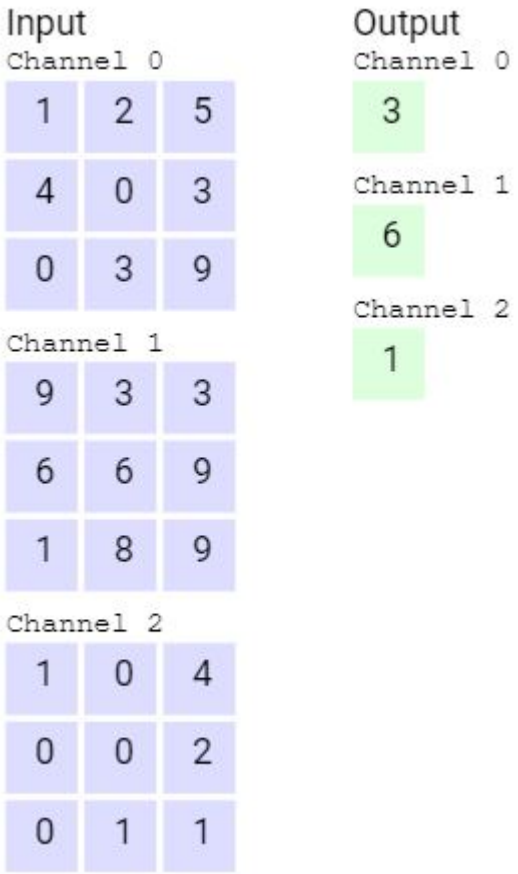
A. Channel-based logits

In the **CNN** section, we applied flattening and a couple of fully-connected layers (with dropout) to obtain our model's logits. However, rather than flattening our data, we can instead use a convolution layer to convert the number of channels in our data to the number of image classes (categories). Then, we apply a special type of average pooling layer to obtain logits for each channel (i.e. image class).

B. Global average pooling

Average pooling follows the exact same concept as max pooling, with the only difference being that the filter performs an *average* rather than a max operation. When the average pooling filter is the same height and width as the input data (i.e. the output is a single average per channel), this is known as *global average pooling*.

We use global average pooling to obtain our per class logits. Each channel corresponds to a unique class, and the channel's averaged value represents the logit for that class.



Global average pooling for an input with 3 channels. Note that our model's average pooling layer operates on 10 channels.

C. Advantages

There are a couple of advantages to using global average pooling rather than fully-connected layers. The first is that global average pooling is more native to the CNN structure, by obtaining logits via channels rather than converting the data to flattened vectors. This allows the CNN to obtain more accurate logits for each image class.

Second, since global average pooling is just a pooling layer, it uses no parameters. This means there is no risk of overfitting at the global average pooling layer. In contrast, fully-connected layers use many weight parameters. Although dropout can help diminish the problem, the risk of overfitting still remains.

Time to Code!

In this chapter you'll be completing the `get_logits` function, which is used at the end of `model_layers` to convert the final convolution layer to logits.

The input to the function, `conv_layer` is a convolution layer applied to the dropout from the previous chapter.

We then use an average pooling layer for global average pooling of `conv_layer`. In TensorFlow, an average pooling layer is applied through the `tf.layers.average_pooling2d` function. The function takes in the exact same arguments as `tf.layers.max_pooling2d`.

For global average pooling, we want the filter size to be the same height and width as the input data. Since the data is in NHWC format, we set the pooling filter size to `[conv_layer.shape[1], conv_layer.shape[2]]`, which represent the H (height) and W (width) dimensions of the data. We use a stride size of `1` because the filter has the same height/width as the data.

Set `avg_pool1` equal to `tf.layers.average_pooling2d` applied with `conv_layer` as the first argument, the filter size as the second argument, and the stride size as the third argument.

The output of global average pooling is still in NHWC format, with height and width of size 1. Since we want the `logits` to be vectors of size `self.output_size`, we need to flatten the data. To do this, we'll use TensorFlow's built-in `tf.layers.flatten` function.

The required argument of `tf.layers.flatten` is an input tensor where the first dimension represents the batch size. The output of `tf.layers.flatten` is the flattened data across the batch. In terms of NHWC, the output has shape `(N, H*W*C)`. The function also takes in a keyword argument for `name`.

Set `logits` equal to `tf.layers.flatten` applied with `avg_pool1` as the first argument and `'logits'` as the `name` argument.
Then return `logits`.

```
import tensorflow as tf

class SqueezeNetModel(object):
    # Model Initialization
    def __init__(self, original_dim, resize_dim, output_size):
        self.original_dim = original_dim
        self.resize_dim = resize_dim
        self.output_size = output_size

    # Convert final convolution layer to logits
    def get_logits(self, conv_layer):
        # CODE HERE
        pass
```

```
    # Convolution layer wrapper
    def custom_conv2d(self, inputs, filters, kernel_size, name):

        return tf.layers.conv2d(
            inputs=inputs,
            filters=filters,
            kernel_size=kernel_size,
            activation=tf.nn.relu,
            padding='same',
            name=name)

    # Max pooling layer wrapper
    def custom_max_pooling2d(self, inputs, name):
        return tf.layers.max_pooling2d(
            inputs=inputs,
            pool_size=[2, 2],
            strides=2,
            name=name)

    # SqueezeNet fire module
    def fire_module(self, inputs, squeeze_depth, expand_depth, name):
        with tf.variable_scope(name):
            squeezed_inputs = self.custom_conv2d(
                inputs,
                squeeze_depth,
                [1, 1],
                'squeeze')
            expand1x1 = self.custom_conv2d(
                squeezed_inputs,
                expand_depth,
                [1, 1],
                'expand1x1')
            expand3x3 = self.custom_conv2d(
                squeezed_inputs,
                expand_depth,
                [3, 3],
                'expand3x3')
            return tf.concat([expand1x1, expand3x3], axis=-1)

    # Utility function for multiple fire modules
    def multi_fire_module(self, layer, params_list):
        for params in params_list:
            layer = self.fire_module(
                layer,
                params[0],
                params[1],
                params[2])
        return layer

    # Model Layers
    # inputs: [batch_size, resize_dim, resize_dim, 3]
    def model_layers(self, inputs, is_training):
        conv1 = self.custom_conv2d(
            inputs,
            64,
            [3, 3],
            'conv1')
        pool1 = self.custom_max_pooling2d(
            conv1,
            'pool1')
        fire_params1 = [
            (32, 64, 'fire1'),
```

```
        (32, 64, 'fire2')
    ]
    multi_fire1 = self.multi_fire_module(
        pool1,
        fire_params1)
    pool2 = self.custom_max_pooling2d(
        multi_fire1,
        'pool2')
    fire_params2 = [
        (32, 128, 'fire3'),
        (32, 128, 'fire4')
    ]
    multi_fire2 = self.multi_fire_module(
        pool2,
        fire_params2)
    dropout1 = tf.layers.dropout(multi_fire2, rate=0.5,
        training=is_training)
    final_conv_layer = self.custom_conv2d(
        dropout1,
        self.output_size,
        [1, 1],
        'final_conv')
    return self.get_logits(final_conv_layer)
```

Sparse Labels

Understand sparse representation of labels.

Chapter Goals:

- Learn how to apply a sparse softmax cross entropy

A. Sparse representation

As mentioned in the Initialization chapter, the CIFAR-10 labels are sparsely represented. This means that, rather than being one-hot vectors, each label is just the index of its corresponding image class. Most datasets will use sparse representation for their labels, since it saves a ton of space compared to one-hot representation (especially if there are many image classes).

For training the model, we use a sparse version of softmax cross entropy. In TensorFlow, this is provided through the `tf.nn.sparse_softmax_cross_entropy_with_logits` function.

Below we show the full code for setting up and training the model:

```
import tensorflow as tf

class SqueezeNetModel(object):
    # __init__ and other functions omitted

    # Set up and run model training
    def run_model_setup(self, inputs, labels):
      logits = self.model_layers(inputs, is_training)
      self.probs = tf.nn.softmax(logits, name='probs')
      self.predictions = tf.argmax(
          self.probs, axis=-1, name='predictions')
      is_correct = tf.equal(
          tf.cast(self.predictions, tf.int32),
          labels)
      is_correct_float = tf.cast(
          is_correct,
          tf.float32)
      self.accuracy = tf.reduce_mean(
          is_correct_float)
      # calculate cross entropy
      cross_entropy = tf.nn.sparse_softmax_cross_entropy_with_logits(
          labels=labels,
```

```
        logits=logits)
    self.loss = tf.reduce_mean(
        cross_entropy)

    adam = tf.train.AdamOptimizer()
    self.train_op = adam.minimize(
        self.loss, global_step=self.global_step)
```

B. Image classification

The code below runs a squeezenet model that has been implemented in the backend. The model was trained on the CIFAR-10 dataset.

It will prompt you to upload your own image, and then print its guess for which of the CIFAR-10 classes your image depicts.

```
run_squeezenet_model()
```

Quiz

1 Why is data augmentation useful?

COMPLETED 0%

1 of 4

Introduction

In this section of the course you will be building ResNet, the winner of the 2015 ImageNet Challenge and one of the major innovations in deep learning. Specifically, you'll recreate Version 2 of ResNet, which switches the ordering of certain layers from Version 1 for better performance.

A. Deep learning obstacles

While deep learning can be a great tool for solving modern day problems, there are still obstacles to overcome when building our models. Since deep models (models with many layers) tend to have numerous weight parameters, they take a long time to train. Furthermore, because deep models extract more hidden features than shallow models, they have a higher risk of overfitting the training data.

Nevertheless, the above two issues can be dealt with relatively seamlessly. Using modern day GPUs and distributed training (i.e. training a model in parallel across multiple processing units), models can be trained in significantly less time. We have also already gone over a few tricks to reduce overfitting, such as max pooling layers, dropout, and global average pooling.

Although issues in deep learning continue to pop up, researchers are usually able to solve them. This problem solving process oftentimes leads to groundbreaking progress and revolutionary discoveries. In fact, the ResNet model that we will be building was created specifically to solve a problem in deep learning known as *degradation*.

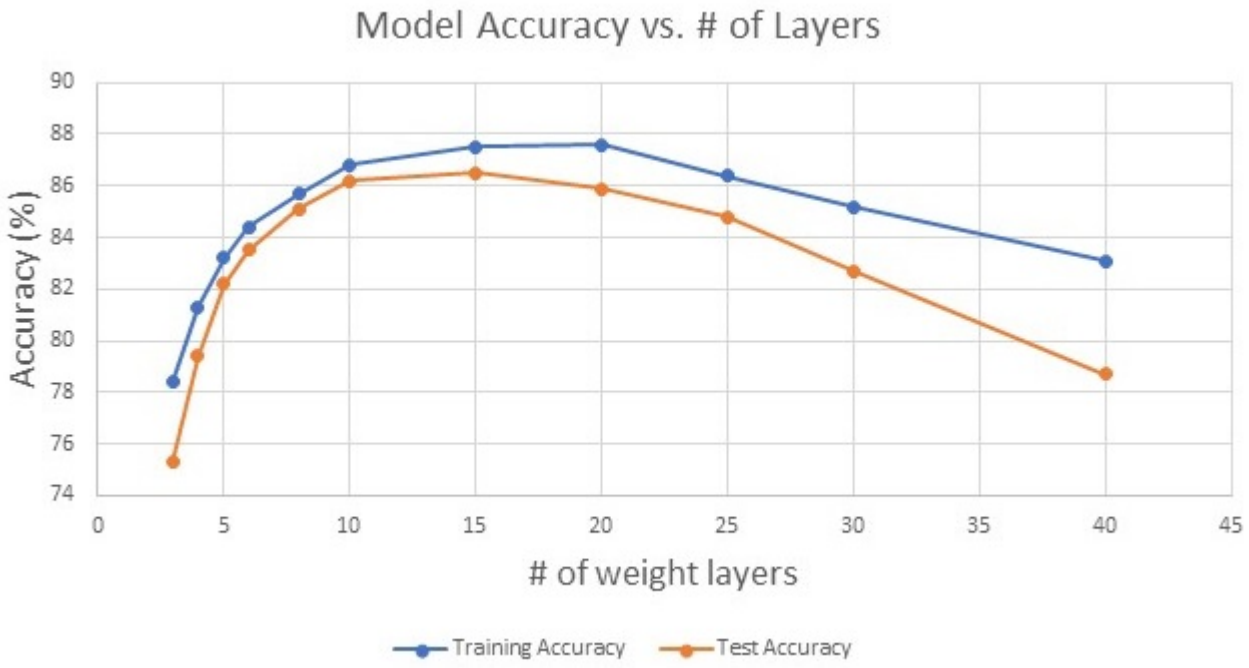
B. Degradation

After the breakthrough made by AlexNet in 2012, people realized that adding more layers to their CNN improved the model's performance. Building off the AlexNet architecture, which used 5 convolution layers and 3 fully-connected layers, researchers began creating models with double digit layers. In 2014,

the top two models in the ImageNet Challenge both had around 20 weight

layers. However, in the process of adding more and more layers to models, the degradation problem was discovered.

Degradation of deep neural networks refers to the plateau, followed by rapid decline, in model accuracy when increasing model depth beyond a certain point.



Example of model degradation. The blue line represents the model accuracy on the training set while the orange line represents accuracy on the test set.

This problem was identified by the creators of ResNet, and they realized it was not caused by overfitting. In overfitting, a model will have incredibly high accuracy on the training set but low accuracy on the test set. The problem of degradation, however, caused the model to have *lower accuracy in both training and testing*.

C. ResNet

The ResNet model architecture was developed specifically to solve the degradation problem. Similar to how the SqueezeNet model (see the **SqueezeNet** section) uses fire modules as its main building block, the ResNet model also has a main building block. Each of its building blocks incorporate *residual learning* to counteract degradation, which we'll discuss more in later chapters.

With the ResNet architecture, we're able to again see model improvement by adding more weight layers. In fact, the ResNet model that won the 2015

ImageNet Challenge had a staggering 152 layers!

ImageNet

Label images using a CNN trained on the ImageNet dataset.

Chapter Goals:

- Learn about the ImageNet dataset

A. ImageNet dataset

The dataset used in the ImageNet Challenge contains 1.4 million images, with 1.2M in the training set, 50,000 in the validation set, and the remaining 150,000 in the (unreleased) test set. The images are distributed across 1,000 categories, which are listed here.

B. Image preprocessing

We first perform image preprocessing and data augmentation. However, while the CIFAR-10 images were all 32x32, the ImageNet images have varying heights and widths.

In order to have some consistency with image sizes, we resize each image such that its new minimum dimension has size equal to some fixed value (`min_aspect_dim`). When resizing, we maintain the image's aspect ratio, i.e. the ratio between its height and width dimensions.

C. Other models

Since the AlexNet model broke the mold in 2012, the ImageNet Challenge has been dominated by deep CNNs. The model that won in 2014 (the year prior to ResNet) was *GoogLeNet*, developed by a team of Google researchers. GoogLeNet is also known as the Inception model due to its *inception module*, which is pretty similar to the fire module of SqueezeNet.

Initialization

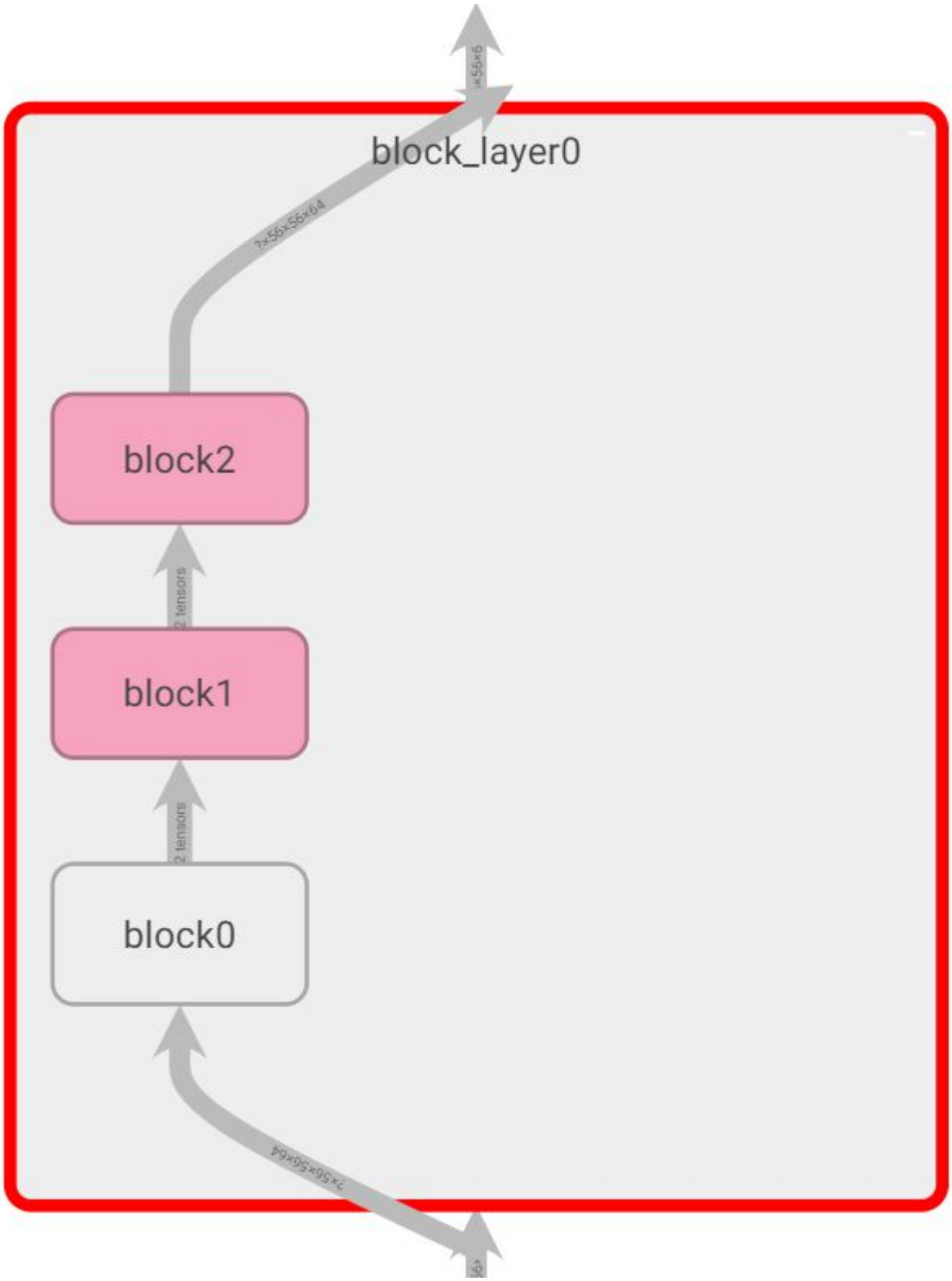
Discover how the ResNet model is structured for each of its configurations.

Chapter Goals:

- Learn about the ResNet model structure
- Understand the difference between channels-first and channels-last

A. Model overview

The ResNet model follows a repetitive structure built around its main building block. The overall model contains four layers of blocks, where the number of blocks per layer depends on how large we want our ResNet model to be. In our code, the `block_layer_sizes` dictionary gives the number of blocks for each of the four block layers, based on the total number of weight layers in the ResNet model.



Structure of a block layer with 3 blocks.

There are six options for how many weight layers (convolution + one fully-connected layer) are in the ResNet model: 18, 34, 50, 101, 152, 200. When the number of layers is less than 50, the model uses a regular building block. However, when the number of layers is 50 or more, the model uses a *bottleneck* block. We'll discuss what a bottleneck block is in a later chapter.

B. Filters and strides

The number of filters used in the model's convolution layers starts off at 64. In each subsequent block layer, we double the number of filters used. This follows the practice of increasing the number of filters for convolution layers deeper in the model.

Most of the blocks in ResNet use a stride size of 1 for their convolution layers. Only the first block of each block layer may use a different stride size. The stride sizes for the first block in each layer are defined in `self.block_strides`.

C. Channel placement

So far, all our model data has been in the NHWC format. This is referred to as *channels-last* placement because the channels are the last dimension. However, TensorFlow also supports *channels-first* placement, i.e. NCHW format. Since the NCHW format is optimized for GPU training and the NHWC format is a bit better when using CPUs, we create our model to allow for both formats.

When we initialize our model, the channel format (`'channels_first'` or `'channels_last'`) is specified through the `data_format` argument. This matches the `data_format` keyword argument that can be used in `tf.layers.conv2d`, `tf.layers.max_pooling2d`, and `tf.layers.average_pooling2d`.

Time to Code!

In this section of the course, you'll be creating the `ResNetModel` class, which represents a ResNet model.

Specifically in this chapter, you'll be completing the initialization of the `ResNetModel`. The `__init__` function is already completed with some variable initialization. Your task will be to set up the parts specific to the ResNet model: the blocks per layer and bottleneck usage.

We'll first set up the number of blocks to use per block layer.

Set `self.block_layer_sizes` equal to `block_layer_sizes[num_layers]`.

Now we set `self.bottleneck` to a boolean, which is `True` if the model uses bottleneck blocks and `False` if it uses regular blocks.

Set `self.bottleneck` equal to `num_layers >= 50`.

```
import tensorflow as tf

block_layer_sizes = {
    18: [2, 2, 2, 2],
    34: [3, 4, 6, 3],
    50: [3, 4, 6, 3],
    101: [3, 4, 23, 3],
    152: [3, 8, 36, 3],
    200: [3, 24, 36, 3]
}

class ResNetModel(object):
    # Model Initialization
    def __init__(self, min_aspect_dim, resize_dim, num_layers, output_size,
        data_format='channels_last'):
        self.min_aspect_dim = min_aspect_dim
        self.resize_dim = resize_dim
        self.filters_initial = 64
        self.block_strides = [1, 2, 2, 2]
        self.data_format = data_format
        self.output_size = output_size
        # CODE HERE
```

Padding

Create custom padding layers to maintain consistency in padding.

Chapter Goals:

- Implement consistent padding based only on kernel size

A. Padding consistency

In the **CNN** section, we mentioned that TensorFlow uses the minimum amount of padding necessary when we set `padding='same'` for our convolution or pooling layers. However, the amount of padding used depends on the kernel size, stride size, and input height/width. This leads to inconsistent padding amounts at different layers of our model.

Since ResNet has so many layers and follows a repetitive structure, we want to maintain as much consistency as possible. Therefore, when we pad our data for a convolution layer, we want the padding to be solely based on the size of the kernel.

Time to Code!

In this chapter you'll be completing the custom padding function, `custom_padding`.

We pad the same number of rows as we do columns, which will be one less than the kernel dimension.

Set `pad_total` equal to one less than `kernel_size`.

We define padding "before" as padding to the left and top of the data, while padding "after" is padding to the right and bottom. Similar to TensorFlow, we'll distribute the padding as evenly as possible.

Set `pad_before` equal to half (rounded down) of `pad_total`.

Set `pad_after` equal to the remaining amount of padding.

The function we use to pad our data is `tf.pad`. The function takes in two required arguments:

- `tensor`: The tensor that we apply padding to.
- `paddings`: A tensor of integers with shape `(num_dims, 2)`, where `num_dims` represents the number of dimensions of the input tensor. Note that `paddings` can also be a list of lists, where the outer list has length `num_dims` and each inner list contains two integers.

Since our data is either NHWC or NCHW format, `num_dims = 4`. However, depending on the channel placement, the `paddings` argument to `tf.pad` can differ slightly. Therefore, we use an `if...else` code block to cover both data formats.

Create an `if...else` with `self.data_format == 'channels_first'` as the `if` condition.

The `paddings` argument will have shape `(4, 2)`, since `num_dims = 4`. We'll use a list of four smaller lists, each containing two integers. Each two integer list corresponds to a specific dimension in our data, with the first integer representing the amount of padding "before" and the second integer representing the amount of padding "after".

Since we only apply padding to the H and W dimensions of our data, we set the padding for the N and C dimensions to `[0, 0]`. The N and C dimensions correspond to indexes 0 and 1 of the `paddings` list in NCHW (channels-first) format. For the H and W dimensions, we set the padding to `[pad_before, pad_after]`.

Inside the `if` block, set `padded_inputs` equal to `tf.pad` with `inputs` as the first argument and the NCHW `paddings` list as the second argument.

For NHWC (channels-last) format, the C dimension is index 3 of the `paddings` list while the H and W dimensions are indexes 1 and 2, respectively. So indexes 0 and 3 of the NHWC `paddings` list will contain `[0, 0]` and indexes 1 and 2 will contain `[pad_before, pad_after]`.

Inside the `else` block, set `padded_inputs` equal to `tf.pad` with `inputs` as

the first argument and the NHWC `paddings` list as the second argument.

```
import tensorflow as tf

block_layer_sizes = {
    18: [2, 2, 2, 2],
    34: [3, 4, 6, 3],
    50: [3, 4, 6, 3],
    101: [3, 4, 23, 3],
    152: [3, 8, 36, 3],
    200: [3, 24, 36, 3]
}

class ResNetModel(object):
    # Model Initialization
    def __init__(self, min_aspect_dim, resize_dim, num_layers, output_size,
        data_format='channels_last'):
        self.min_aspect_dim = min_aspect_dim
        self.resize_dim = resize_dim
        self.filters_initial = 64
        self.block_strides = [1, 2, 2, 2]
        self.data_format = data_format
        self.output_size = output_size
        self.block_layer_sizes = block_layer_sizes[num_layers]
        self.bottleneck = num_layers >= 50

    # Custom padding function
    def custom_padding(self, inputs, kernel_size):
        # CODE HERE
        pass

    # Custom convolution function w/ consistent padding
    def custom_conv2d(self, inputs, filters, kernel_size, strides, name=None):
        if strides > 1:
            padding = 'valid'
            inputs = self.custom_padding(inputs, kernel_size)
        else:
            padding = 'same'
        return tf.layers.conv2d(
            inputs=inputs, filters=filters, kernel_size=kernel_size,
            strides=strides, padding=padding, data_format=self.data_format,
            name=name)
```

Pre-activation

Learn why batch normalization and pre-activation are important in ResNet.

Chapter Goals:

- Learn about internal covariate shift and batch normalization
- Understand the difference between pre-activation and post-activation
- Implement the pre-activation function

A. Internal covariate shift

When training models with many weight layers, e.g. ResNet, a problem known as *internal covariate shift* can occur. To understand this problem, we first define a related problem known as *covariate shift*.

A covariate shift occurs when the input data's *distribution* changes and the model cannot handle the change properly. An example would be if a model was trained to classify between different dog breeds, with a training set of only images of brown dogs. Then if we test the model on images of yellow dogs, the performance may not be as good as we expected.

In this case, the model's original input distribution was limited to just brown dogs, and changing the input distribution to a different color of dogs introduced covariate shift.

An internal covariate shift is essentially just a covariate shift that happens between layers of a model. Since the input of one layer is the output of the previous layer, the input distribution for a layer is the same as the output distribution of the previous layer.

Because the output distribution of a layer depends on its weights, and the weights of a model are constantly being updated, each layer's output distribution will constantly change (though by incremental amounts). In a model without many layers, the incremental changes in layer distributions don't really have much impact. However, in models with many layers these

incremental changes will eventually add up, and lead to internal covariate shift at deeper layers.

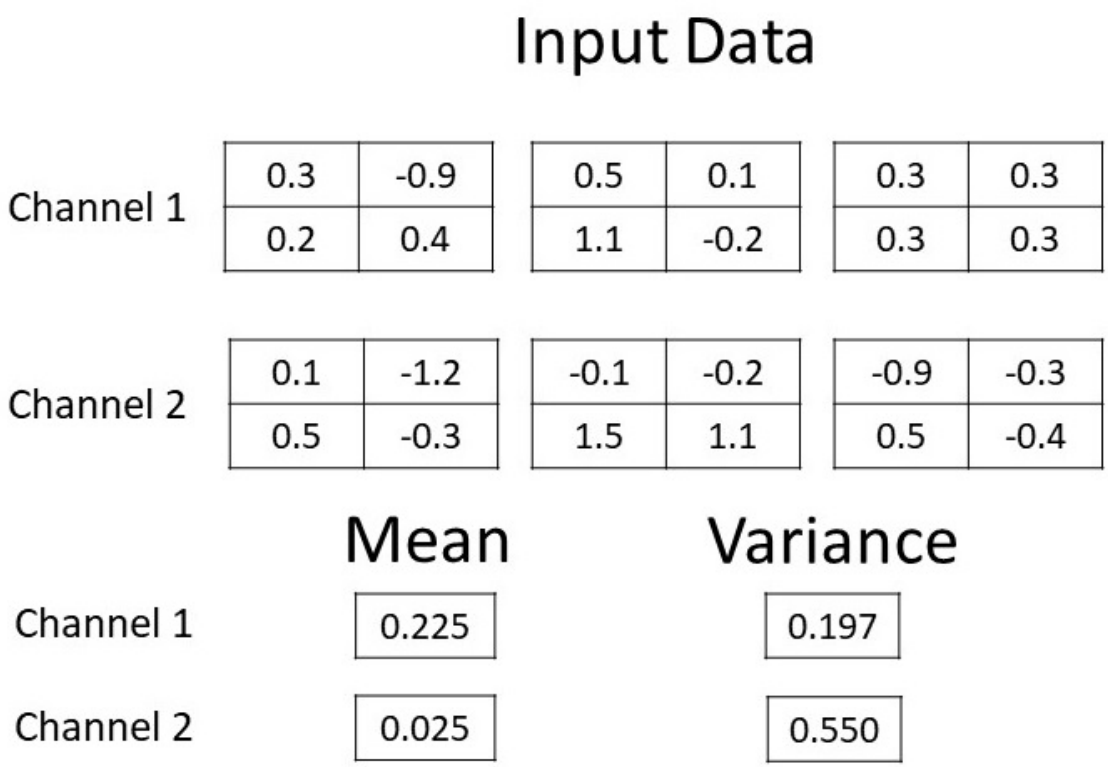
B. Batch normalization

The solution to internal covariate shift is batch normalization. Since internal covariate shift is caused by distribution changes between layers, we can remedy this by enforcing a fixed distribution to the inputs of each layer.

Batch normalization accomplishes this by subtracting the mean from the inputs and dividing by the standard deviation (i.e. square-root variance). This results in a standardized distribution (i.e. mean of 0 and variance of 1).

One thing to note is that batch normalization is applied across a specific dimension of the data. For CNNs, we apply it across the channels dimension, meaning we standardize the values for each channel of the input data. So the mean and variance are actually vectors of `num_channels` values.

Below we show standardization of an input with 2 channels and a batch size of 3.



Input data (mean and variance per channel are shown)

Standardized Data

Channel 1	0.17	-2.53	0.62	-0.28	0.17	0.17
	-0.06	0.39	1.97	-0.96	0.17	0.17
Channel 2	0.10	-1.65	-0.17	-0.30	-1.25	-0.44
	-0.64	-0.44	1.99	1.45	0.64	-0.57

Standardized data (for the data shown above)

For some layers we might not want a standardized distribution of the inputs. Maybe we want a distribution with a different mean or variance. Luckily, batch normalization has two trainable variables, γ and β, that allow us to change the variance and mean, respectively, of the distribution.

$$\mathrm{BN}(\mathbf{x}) = \gamma * \hat{\mathbf{x}} + \beta$$

The above formula represents batch normalization (BN) applied to input data $\mathbf{x}$. Note that $\hat{\mathbf{x}}$ represents the standardized input data.

Since γ and β are trainable variables, the model will automatically fine-tune their values for each batch normalization layer. In fact, if γ is set to the original standard deviation and β is set to the original mean of the inputs, batch normalization will produce the original input data values.

C. Moving average

During training, batch normalization calculates the mean and variance of its inputs at every training step. To obtain the best possible results during evaluation, we maintain an exponential moving average of the mean and variance vectors for each batch normalization layer. So if our model has m batch normalization layers, we maintain the mean and variance per batch normalization layer, for a total of m means and m variances.

During evaluation, we use the average mean and variance for each batch normalization layer rather than the input data's mean and variance. The average mean and variance provide a better estimate for the overall dataset,

which helps our model's accuracy.

At the bottom of the `run_model_setup` function, you can see that we added in some code involving an `update_ops` variable. The code tells our model to update the moving average and variance for each batch normalization layer after every training step.

D. Pre-activation

When we use batch normalization, we apply it right before an activation function, e.g. ReLU. Normally, the activation function in CNNs comes after each convolution layer. This is known as *post-activation*. Version 1 of ResNet used the traditional method of post-activation, meaning its convolution layers would be followed by a batch normalization, which would then be followed by the ReLU activation.

However, Version 2 of ResNet switched up the ordering so that the batch normalization and activation would now come *before* the convolution layer. This is known as *pre-activation*. Pre-activation has been shown to work better with ResNet, which is why we are using it in our code.

You may have noticed that the `custom_conv2d` function didn't use the `activation` keyword when setting `tf.layers.conv2d`. This was purposely done so we could manually apply ReLU activation prior to the convolution layer, rather than letting TensorFlow apply it right after.

Time to Code!

In this chapter you'll be completing the `pre_activation` function, which applies the pre-activation described above.

First we'll set the channels axis (dimension) depending on the data format.

Set `axis` equal to `1` if `self.data_format` is `'channels_first'`, otherwise set it equal to `3`.

The first step to pre-activation is applying batch normalization to the input data. In TensorFlow, the function we use for batch normalization is `tf.layers.batch_normalization`. The only required argument for the function is the input data. However, there are a ton of keyword arguments, most of which will rarely be changed from their default value.

The keyword arguments that we change are `axis` and `training`. The `axis` argument represents the axis (dimension) we apply batch normalization to. In our code, we apply it to the channels dimension. The `training` argument represents whether the model is training or not, since batch normalization is applied differently for training and evaluation.

Set `bn_inputs` equal to `tf.layers.batch_normalization` with `inputs` as the first argument, along with keyword arguments `axis=axis` and `training=is_training`.

We finish the pre-activation by applying the ReLU function to our batch normalized input data.

Set `pre_activated_inputs` equal to `tf.nn.relu` applied to `bn_inputs`. Then return `pre_activated_inputs`.

```
import tensorflow as tf

# block_layer_sizes loaded in backend

class ResNetModel(object):
    # Model Initialization
    def __init__(self, min_aspect_dim, resize_dim, num_layers, output_size,
        data_format='channels_last'):
        self.min_aspect_dim = min_aspect_dim
        self.resize_dim = resize_dim
        self.filters_initial = 64
        self.block_strides = [1, 2, 2, 2]
        self.data_format = data_format
        self.output_size = output_size
        self.block_layer_sizes = block_layer_sizes[num_layers]
        self.bottleneck = num_layers >= 50

    # Applies pre-activation to the inputs
    def pre_activation(self, inputs, is_training):
        # CODE HERE
        pass
```

Shortcut

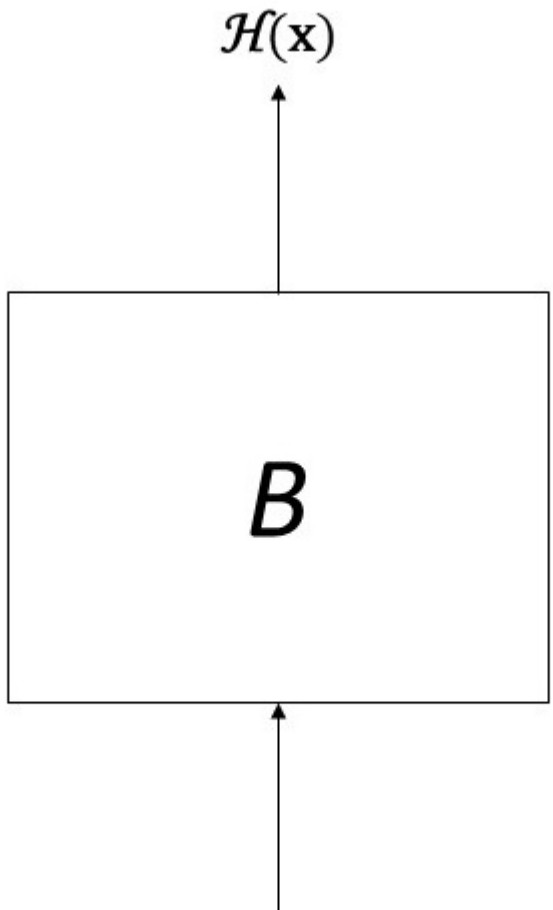
Understand how shortcuts can improve performance in large models.

Chapter Goals:

- Learn about mapping functions and identity mapping
- Understand the purpose of a shortcut for residual learning
- Implement a wrapper for the pre-activation function that also returns the shortcut

A. Mapping functions

To understand the intuition behind ResNet, we need to discuss its building blocks. Each ResNet building block takes in an input, $\mathbf{x}$, and produces some output $\mathcal{H}(\mathbf{x})$, where $\mathcal{H}$ represents the block's *mapping function*. The mapping function, $\mathcal{H}$, is a mathematical representation of the block itself; it takes in an input and, using the weights within the block, produces an output.



$\mathbf{x}$

The image above shows the input and output for block B with mapping function $\mathcal{H}$.

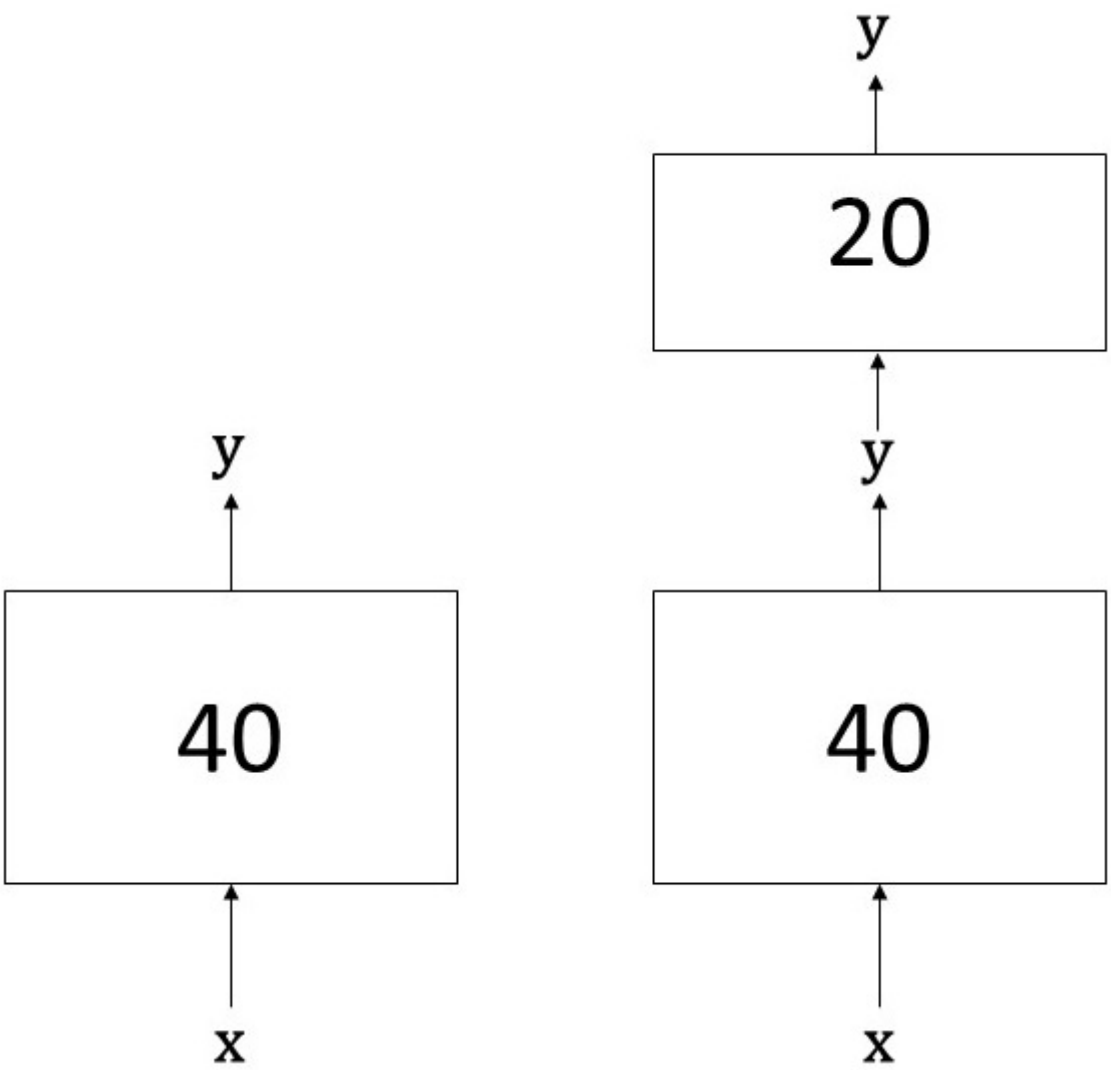
For now, it suffices to know that a block is just a stack of convolution layers (we'll discuss the inner workings of a block in the next two chapters).

B. Identity mapping

We've previously discussed the issue of *degradation*, where a model's performance decreases after a certain number of layers are added. Despite its effects, degradation is actually a pretty counter-intuitive problem.

Let's say our model performs well at 40 convolution layers, so we want to add 20 more convolution layers (60 layers total). There is a simple way for the larger model to achieve the same performance as the smaller model: use the same weights as the smaller model for the first 40 layers, then make the last 20 convolution layers an *identity mapping*.

An identity mapping just means the output for a layer (or set of layers, e.g. a ResNet block) is the same as the input. So if the larger model used the same weights as the smaller model, followed by an identity mapping, its outputs would be identical to those of the smaller model.



The image above shows a 40 layer model (left) takes in input $\mathbf{x}$ and produces $\mathbf{y}$. 60 layer model (right) also produces $\mathbf{y}$ by making its last 20 layers an identity mapping.

However, for some reason the larger model is worse, despite having a simple solution to at least match the smaller model's performance. It turns out that training a model's layers to learn identity mappings is actually pretty difficult.

C. Residual learning

Let's say for some block B, its optimal mapping function is $\mathcal{H}_B$. If the optimal function is close to an identity mapping (i.e. $\mathcal{H}_B(\mathbf{x}) = \mathbf{x}$), learning the function will be a hard task. Instead, we can have the block learn a different mapping function, $\mathcal{F}_B$, such that

$$\mathcal{F}_B(\mathbf{x}) = \mathcal{H}_B(\mathbf{x}) - \mathbf{x}$$

We also add the input, $\mathbf{x}$, to the block's output. This is referred to as a *shortcut*

We also add the input, $\mathbf{x}$, to the block's output. This is referred to as a *skip connection*, or just *shortcut*. So the modified block's output will now be

$\mathcal{F}_B(\mathbf{x}) + \mathbf{x}$, which is equivalent to the optimal function, $\mathcal{H}_B(\mathbf{x})$. The process of learning $\mathcal{F}_B(\mathbf{x})$ is known as *residual learning*. When the optimal mapping function, $\mathcal{H}_B(\mathbf{x})$, is close to the identity mapping, it is much easier for a block to learn $\mathcal{F}_B(\mathbf{x})$ than $\mathcal{H}_B(\mathbf{x})$. In the next chapter, we'll explain why.

D. Projection shortcut

When a block uses a stride size greater than 1 for its convolution layers, its output will have reduced height and width compared to its input. Since the shortcut is the same as the input data, it cannot be added to the block output (addition requires equal dimension sizes). In this case, we use a *projection shortcut*.

A projection shortcut is the result of applying a convolution layer, with 1x1 kernels, to the pre-activated input data. This convolution layer ensures that the shortcut has the same dimensions as the block's output, by using the same stride size and number of filters.

We apply the convolution layer to the pre-activated input data, rather than directly to the input data, for consistency with the rest of the block (where each convolution layer is preceded by pre-activation).

Time to Code!

In this chapter you'll complete the `pre_activation_with_shortcut` function (line **53**), which applies pre-activation to input data and also returns a shortcut.

The function already has the pre-activated input and shortcut configuration set up. Your task is to write code inside the `if` block, which sets the shortcut to a convolution layer on top of the pre-activated inputs.

The stride length for the convolution layer is given in `shortcut_params[1]`.

Set `strides` equal to the second element of `shortcut_params`.

The convolution layer will use 1x1 kernels, so we can just set `kernel_size` to 1 (single integer means same width and height for the kernel).

Set `shortcut` equal to `self.custom_conv2d` with `pre_activated_inputs` as the

Set `shortcut` equal to `self.custom_conv2d` with `pre_activated_inputs` as the first argument, `shortcut_filters` as the number of filters, a kernel size of `1`, and `strides` as the stride size.

```
import tensorflow as tf

# block_layer_sizes loaded in backend

class ResNetModel(object):
    # Model Initialization
    def __init__(self, min_aspect_dim, resize_dim, num_layers, output_size,
        data_format='channels_last'):
        self.min_aspect_dim = min_aspect_dim
        self.resize_dim = resize_dim
        self.filters_initial = 64
        self.block_strides = [1, 2, 2, 2]
        self.data_format = data_format
        self.output_size = output_size
        self.block_layer_sizes = block_layer_sizes[num_layers]
        self.bottleneck = num_layers >= 50

    # Applies consistent padding to the inputs
    def custom_padding(self, inputs, kernel_size):
        pad_total = kernel_size - 1
        pad_before = pad_total // 2
        pad_after = pad_total - pad_before
        if self.data_format == 'channels_first':
            padded_inputs = tf.pad(
                inputs,
                [[0, 0], [0, 0], [pad_before, pad_after], [pad_before, pad_after]])
        else:
            padded_inputs = tf.pad(
                inputs,
                [[0, 0], [pad_before, pad_after], [pad_before, pad_after], [0, 0]])
        return padded_inputs

    # Customized convolution layer w/ consistent padding
    def custom_conv2d(self, inputs, filters, kernel_size, strides, name=None):
        if strides > 1:
            padding = 'valid'
            inputs = self.custom_padding(inputs, kernel_size)
        else:
            padding = 'same'
        return tf.layers.conv2d(
            inputs=inputs, filters=filters, kernel_size=kernel_size,
            strides=strides, padding=padding, data_format=self.data_format,
            name=name)

    # Apply pre-activation to input data
    def pre_activation(self, inputs, is_training):
        axis = 1 if self.data_format == 'channels_first' else 3
        bn_inputs = tf.layers.batch_normalization(inputs, axis=axis, training=is_training)
        pre_activated_inputs = tf.nn.relu(bn_inputs)
        return pre_activated_inputs

    # Returns pre-activated inputs and the shortcut
    def pre_activation_with_shortcut(self, inputs, is_training, shortcut_params):
        pre_activated_inputs = self.pre_activation(inputs, is_training)
        shortcut = inputs
```

```
        shortcut_filters = shortcut_params[0]
        if shortcut_filters is not None:
            # CODE HERE

            pass
        return pre_activated_inputs, shortcut
```

ResNet Block

Learn about the main building block of ResNet and how it works.

Chapter Goals:

- Understand the purpose of residual learning
- Learn the structure of a ResNet building block
- Create a function for a ResNet building block

A. Learning the identity

In the previous chapter, we touched upon *residual learning*, i.e. the process of a block learning the *residual function*:

$$\mathcal{F}_B(\mathbf{x}) = \mathcal{H}_B(\mathbf{x}) - \mathbf{x}$$

where $\mathcal{H}_B(\mathbf{x})$ is the optimal mapping function.

You might ask why we use residual learning, since the residual function is pretty similar to the optimal function. The reason is because it is much easier for a block to learn a *zero mapping* than an identity mapping.

A zero mapping simply returns an output of all 0's. This is very easy to learn; the block can just be trained so that all its weights are 0. However, if the block learns a zero mapping, i.e. $\mathcal{F}_B(\mathbf{x}) = \mathbf{0}$, then the modified output (shortcut added) becomes

$$\mathcal{F}_B(\mathbf{x}) + \mathbf{x} = \mathbf{0} + \mathbf{x} = \mathbf{x}$$

That's exactly the identity mapping! So in learning the easy zero mapping, we've created a block that corresponds to an identity mapping, which is a much harder task.

B. Improving performance

We can now add many layers to a model and still avoid degradation, by having the additional layers represent identity mappings (which is possible

due to residual learning). What's more amazing, though, is that residual learning actually *increases* model performance when adding many layers.

Let's say we have a ResNet model with 20 blocks, which already has really good performance. We'll add 5 more blocks to the model and train it with residual learning. If the first 20 blocks of our larger model learn the same weights as the smaller model, the 5 additional blocks don't need to modify the output of the first 20 blocks by much, since the smaller model was already really good.

In other words, we want each of our 5 additional blocks to learn a mapping function relatively close to the identity mapping. That's exactly what residual learning is designed to do, i.e. allow additional blocks to make small tweaks to improve model performance.

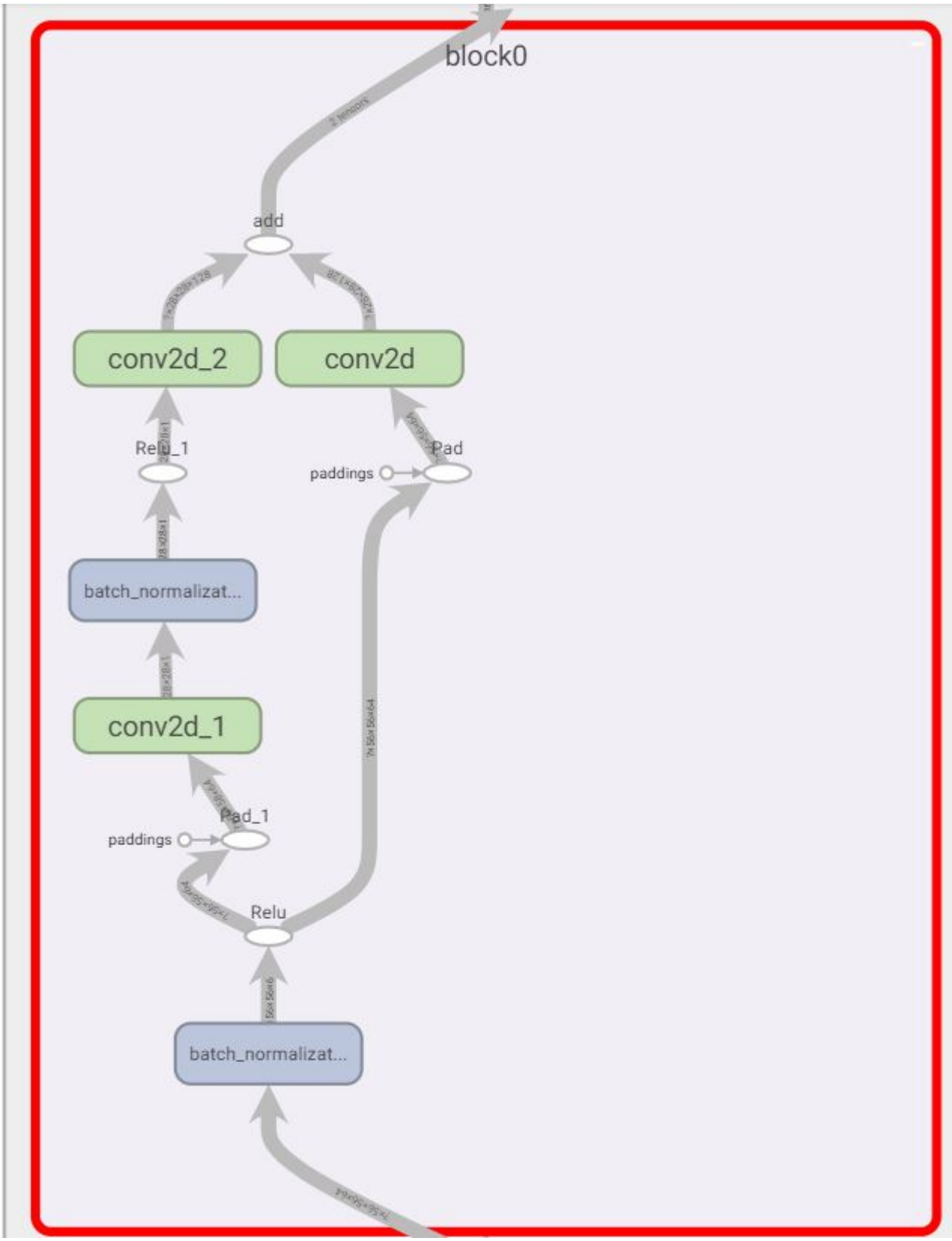
C. Structure of a block

We've talked a lot about blocks in the context of ResNet and residual learning. Now it's time to get into the actual structure of a block.

As mentioned in the previous chapter, a block is made up of multiple convolution layers. Specifically, it uses two convolution layers, both with 3x3 kernels and pre-activated inputs.

The second convolution layer always uses stride size of 1, so the input and output have the same height and width dimensions. The first convolution layer will also mainly use a stride size of 1. The only times it uses a stride size of 2 is for dimension reduction purposes.

Since a ResNet block uses residual learning, the shortcut is added to the block's residual output, $\mathcal{F}_B(\mathbf{x})$, to produce the overall output.



Structure of a ResNet block, with a projection shortcut. The left arrow path represents the block's structure, while the right arrow path represents the projection shortcut.

Time to Code!

In this chapter you'll complete the `regular_block` function (line **63**), which creates a ResNet building block.

The function creates the building block inside a *variable scope* (`with` block),

which helps organize the model (similar to how a directory organizes files).

Inside the variable scope, code has already been filled in to create the first convolution layer and shortcut.

Your task is to create the second convolution layer, then return the output of the building block.

Set `pre_activated2` equal to `self.pre_activation` with `conv1` as the first argument and `is_training` as the second argument.

We use `pre_activated2` as the input to the second convolution layer. The second convolution layer will also use `filters` for the number of filters and a `kernel_size` of 3. This time, the stride size will be 1.

Set `conv2` equal to `self.custom_conv2d` with the specified input arguments.

The output of the building block is the sum of the shortcut and the output of the second convolution layer.

Return the sum of `conv2` and `shortcut`, still inside the `with` block.

```
import tensorflow as tf

# block_layer_sizes loaded in backend

class ResNetModel(object):
    # Model Initialization
    def __init__(self, min_aspect_dim, resize_dim, num_layers, output_size,
        data_format='channels_last'):
        self.min_aspect_dim = min_aspect_dim
        self.resize_dim = resize_dim
        self.filters_initial = 64
        self.block_strides = [1, 2, 2, 2]
        self.data_format = data_format
        self.output_size = output_size
        self.block_layer_sizes = block_layer_sizes[num_layers]
        self.bottleneck = num_layers >= 50

    # Applies consistent padding to the inputs
    def custom_padding(self, inputs, kernel_size):
        pad_total = kernel_size - 1
        pad_before = pad_total // 2
        pad_after = pad_total - pad_before
        if self.data_format == 'channels_first':
            padded_inputs = tf.pad(
                inputs,
                [[0, 0], [0, 0], [pad_before, pad_after], [pad_before, pad_after]])
        else:
            padded_inputs = tf.pad(
                inputs,
```

```
                [[0, 0], [pad_before, pad_after], [pad_before, pad_after], [0, 0]])
        return padded_inputs


    # Customized convolution layer w/ consistent padding
    def custom_conv2d(self, inputs, filters, kernel_size, strides, name=None):
        if strides > 1:
            padding = 'valid'
            inputs = self.custom_padding(inputs, kernel_size)
        else:
            padding = 'same'
        return tf.layers.conv2d(
            inputs=inputs, filters=filters, kernel_size=kernel_size,
            strides=strides, padding=padding, data_format=self.data_format,
            name=name)

    # Apply pre-activation to input data
    def pre_activation(self, inputs, is_training):
        axis = 1 if self.data_format == 'channels_first' else 3
        bn_inputs = tf.layers.batch_normalization(inputs, axis=axis, training=is_training)
        pre_activated_inputs = tf.nn.relu(bn_inputs)
        return pre_activated_inputs

    # Returns pre-activated inputs and the shortcut
    def pre_activation_with_shortcut(self, inputs, is_training, shortcut_params):
        pre_activated_inputs = self.pre_activation(inputs, is_training)
        shortcut = inputs
        shortcut_filters = shortcut_params[0]
        if shortcut_filters is not None:
            strides = shortcut_params[1]
            shortcut = self.custom_conv2d(pre_activated_inputs, shortcut_filters, 1, strides)
        return pre_activated_inputs, shortcut

    # ResNet building block
    def regular_block(self, inputs, filters, strides, is_training, index, shortcut_filters=No
        with tf.variable_scope('regular_block{}'.format(index)):
            shortcut_params = (shortcut_filters, strides)
            pre_activated1, shortcut = self.pre_activation_with_shortcut(inputs, is_training,
            conv1 = self.custom_conv2d(pre_activated1, filters, 3, strides)
            # CODE HERE
```

Bottleneck

Understand how bottleneck blocks decrease memory usage for large ResNet models.

Chapter Goals:

- Learn about the bottleneck block and why it's used
- Implement a function for a bottleneck block

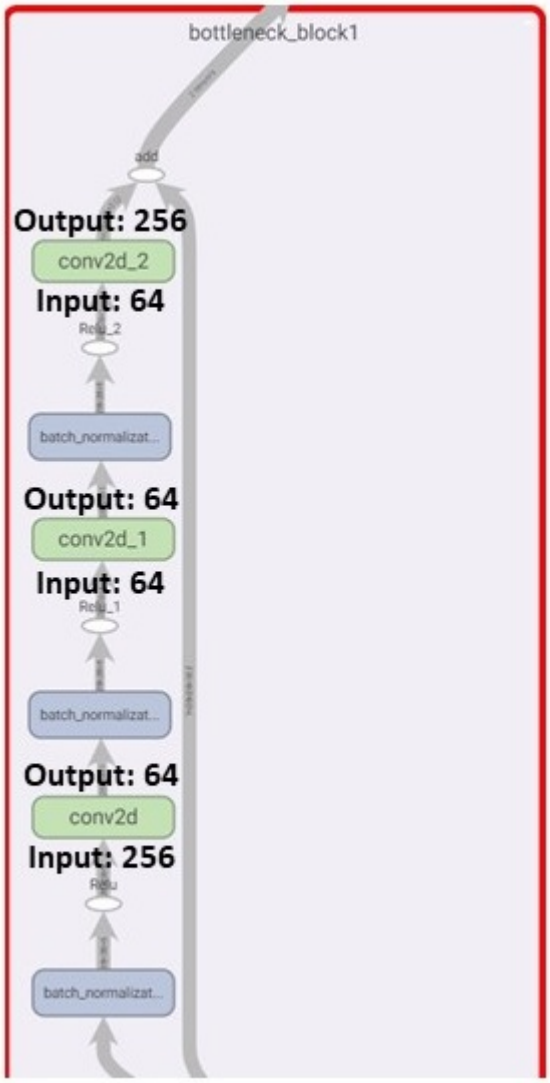
A. Size considerations

The ResNet block we described in the previous chapter is the main building block for models with fewer than 50 layers. Once we hit 50 or more layers, we want to take advantage of the large model depth, and utilize more filters in each convolution layer. However, using more filters results in added weight parameters, which can lead to incredibly long training time.

To counter this, ResNet incorporates the same squeeze and expand concept used by the SqueezeNet fire module. The ResNet blocks for 50+ layer models will now use 3 convolution layers rather than 2, where the first convolution layer squeezes the number of channels in the data and the third convolution layer expands the number of channels. We refer to these blocks as *bottleneck* blocks.

B. Bottleneck block

The third convolution layer of a bottleneck block uses four times as many filters as a regular ResNet block. This means that the input to bottleneck blocks will have four times as many channels (remember that the output of one block is the input to the next). Hence, the first convolution layer acts as a squeeze layer, to reduce the number of channels back to the regular amount.



Example: A bottleneck block (with a shortcut), for input data with 256 channels. The number of channels for the input and output of each convolution layer are labeled in the diagram.

Another similarity to the SqueezeNet fire module is the mixed usage of 1x1 and 3x3 kernels. The bottleneck block uses 1x1 kernels for the first and third convolution layers, while the middle convolution layer still uses 3x3 kernels. This helps reduce the number of weight parameters while still maintaining good performance.

C. Parameter comparison

In the SqueezeNet Lab, we introduced the equation to calculate the number of weight parameters in a convolution layer:

$$P = H_K \times W_K \times F \times C + F$$

where $H_K \times W_K$ is the kernel dimensions, F is the number of filters, and C is the number of input channels.

For a regular ResNet block with 64 filters that takes in an input with 64 channels, the number of weight parameters, (P), used in the block is

$$P_1 = P_2 = 3 \times 3 \times 64 \times 64 + 64 = 36{,}928$$

$$\mathbf{P = P_1 + P_2 = 73{,}856}$$

where P_1 and P_2 represent the number of weight parameters used in the first and second convolution layers, respectively.

For a bottleneck block with 64 filters, the input will have 256 channels. Therefore, the number of weight parameters, P_B, used in the block is

$$P_1 = 1 \times 1 \times 64 \times 256 + 64 = 16{,}448$$

$$P_2 = 3 \times 3 \times 64 \times 64 + 64 = 36{,}928$$

$$P_3 = 1 \times 1 \times 256 \times 64 + 256 = 16{,}640$$

$$\mathbf{P_B = P_1 + P_2 + P_3 = 70{,}016}$$

where P_1, P_2, and P_3 represent the number of weight parameters used in the first, second, and third convolution layers, respectively.

So despite the input and output having four times as many filters, the bottleneck block actually uses fewer weight parameters. This allows us to create very deep ResNet models (e.g. 200 layers) and still be able to train them in a reasonable amount of time.

Time to Code!

In this chapter, you'll complete the `bottleneck_block` function (line **63**), which creates a ResNet bottleneck block.

The function creates the bottleneck block inside a *variable scope* (`with` block), which helps organize the model (similar to how a directory organizes files). Inside the variable scope, code has already been filled in to create the first two convolution layers and shortcut.

Your task is to create the third convolution layer, then return the output of the building block.

Set `pre_activated3` equal to `self.pre_activation` with `conv2` as the first argument and `is_training` as the second argument.

argument and `is_training` as the second argument.

We use `pre_activated3` as the input to the third convolution layer. The convolution layer will use four times as many filters as the previous two convolution layers, and the same kernel size and stride size as the first convolution layer.

Set `conv3` equal to `self.custom_conv2d` applied with the specified input arguments.

The output of the building block is the sum of the shortcut and the output of the third convolution layer.

Return the sum of `conv3` and `shortcut`, still inside the `with` block.

```
import tensorflow as tf

# block_layer_sizes loaded in backend

class ResNetModel(object):
    # Model Initialization
    def __init__(self, min_aspect_dim, resize_dim, num_layers, output_size,
        data_format='channels_last'):
        self.min_aspect_dim = min_aspect_dim
        self.resize_dim = resize_dim
        self.filters_initial = 64
        self.block_strides = [1, 2, 2, 2]
        self.data_format = data_format
        self.output_size = output_size
        self.block_layer_sizes = block_layer_sizes[num_layers]
        self.bottleneck = num_layers >= 50

    # Applies consistent padding to the inputs
    def custom_padding(self, inputs, kernel_size):
        pad_total = kernel_size - 1
        pad_before = pad_total // 2
        pad_after = pad_total - pad_before
        if self.data_format == 'channels_first':
            padded_inputs = tf.pad(
                inputs,
                [[0, 0], [0, 0], [pad_before, pad_after], [pad_before, pad_after]])
        else:
            padded_inputs = tf.pad(
                inputs,
                [[0, 0], [pad_before, pad_after], [pad_before, pad_after], [0, 0]])
        return padded_inputs

    # Customized convolution layer w/ consistent padding
    def custom_conv2d(self, inputs, filters, kernel_size, strides, name=None):
        if strides > 1:
            padding = 'valid'
            inputs = self.custom_padding(inputs, kernel_size)
        else:
            padding = 'same'
```

```
        padding = 'same'
        return tf.layers.conv2d(
            inputs=inputs, filters=filters, kernel_size=kernel_size,
            strides=strides, padding=padding, data_format=self.data_format,
            name=name)

    # Apply pre-activation to input data
    def pre_activation(self, inputs, is_training):
        axis = 1 if self.data_format == 'channels_first' else 3
        bn_inputs = tf.layers.batch_normalization(inputs, axis=axis, training=is_training)
        pre_activated_inputs = tf.nn.relu(bn_inputs)
        return pre_activated_inputs

    # Returns pre-activated inputs and the shortcut
    def pre_activation_with_shortcut(self, inputs, is_training, shortcut_params):
        pre_activated_inputs = self.pre_activation(inputs, is_training)
        shortcut = inputs
        shortcut_filters = shortcut_params[0]
        if shortcut_filters is not None:
            strides = shortcut_params[1]
            shortcut = self.custom_conv2d(pre_activated_inputs, shortcut_filters, 1, strides)
        return pre_activated_inputs, shortcut

    # ResNet bottleneck block
    def bottleneck_block(self, inputs, filters, strides, is_training, index, shortcut_filters
        with tf.variable_scope('bottleneck_block{}'.format(index)):
            shortcut_params = (shortcut_filters, strides)
            pre_activated1, shortcut = self.pre_activation_with_shortcut(inputs, is_training,
            conv1 = self.custom_conv2d(pre_activated1, filters, 1, 1)
            pre_activated2 = self.pre_activation(conv1, is_training)
            conv2 = self.custom_conv2d(pre_activated2, filters, 3, strides)
            # CODE HERE
```

Block Layer

Stack multiple blocks together to create a block layer in ResNet.

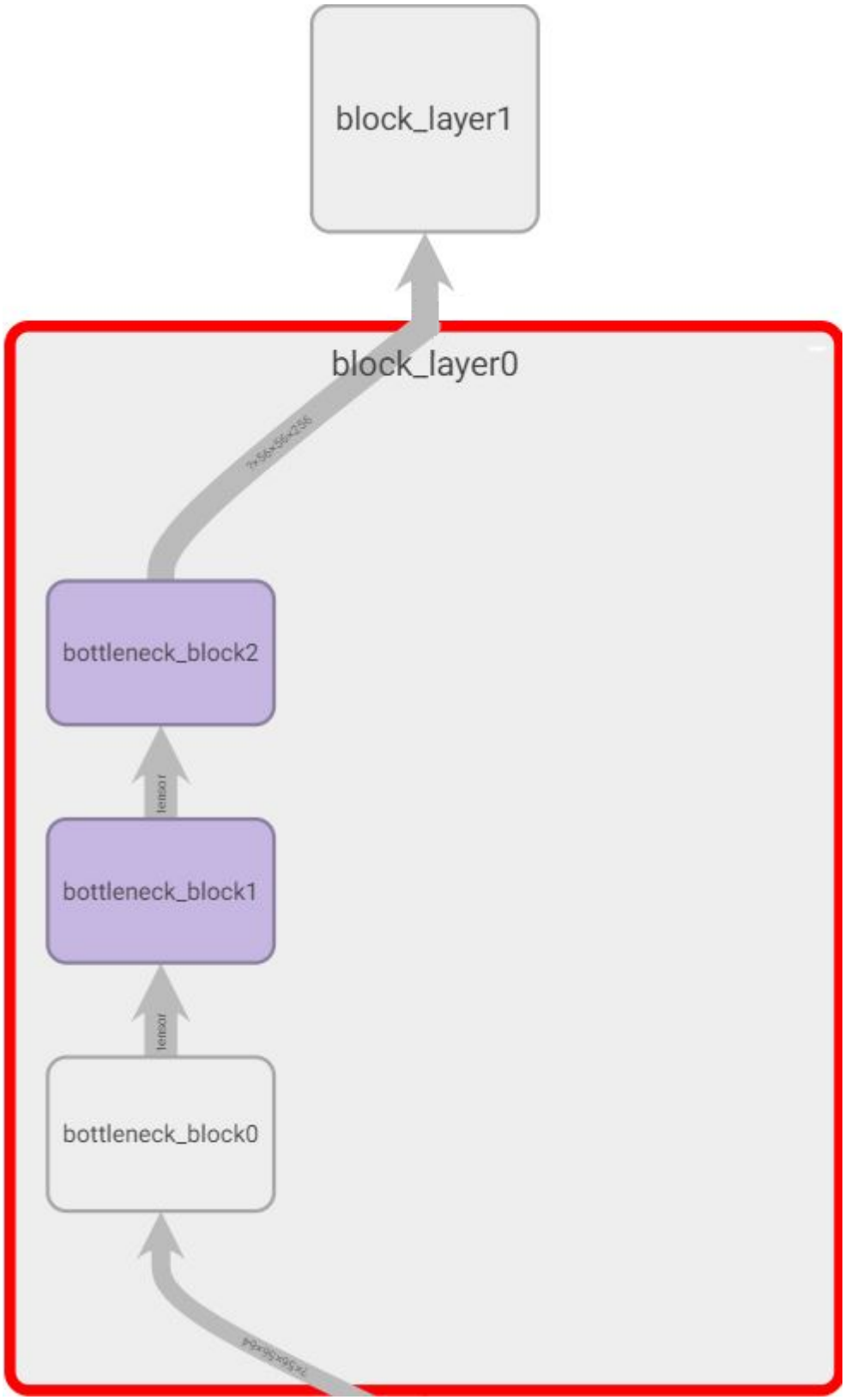
Chapter Goals:

- Learn how a layer of blocks is organized
- Understand the function for creating a layer of blocks

A. Block layers

A ResNet model is made up of four block layers. Each block layer will contain a different number of blocks, depending on the total number of weight layers in the ResNet model. For example, an 18 layer ResNet model has 2 blocks in each block layer.

The blocks within each block layer are connected, so the output of block i is the input of block $i+1$ in the same layer. Furthermore, the four block layers themselves are connected, so the output of block layer j becomes the input of block layer $j+1$.



First block layer of a ResNet model with 50 weight layers. The first bottleneck block uses a projection shortcut, so it is colored differently. The output of the first block layer becomes the input of the second block layer.

The block layers account for a large majority of the weight layers in a ResNet model. However, they don't make up the entire structure of a ResNet model. The block layer is implemented in the `ResNetModel` class as the `block_layer` function:

```
import tensorflow as tf

class ResNetModel(object):
    # __init__ and other functions omitted

    # Creates a layer of blocks
    def block_layer(self, inputs, filters, strides, num_blocks, is_training, index):
        with tf.variable_scope('block_layer{}'.format(index)):
            shortcut_filters = 4 * filters if self.bottleneck else filters
            block_fn = self.bottleneck_block if self.bottleneck else self.regular_block
            block_output = block_fn(inputs, filters, strides, is_training, 0,
                shortcut_filters=shortcut_filters)
            # stack the blocks in this layer
            for i in range(1, num_blocks):
                block_output = block_fn(block_output, filters, 1, is_training, i)
            return block_output
```

As you can see, the function stacks multiple building blocks. The particular block to use (`block_fn`) depends on whether or not the model uses bottlenecks.

Model Layers

Learn about the initial and final layers for ResNet.

Chapter Goals:

- Understand how the ResNet architecture is structured

A. Initial layers

Prior to the block layers, the ResNet model uses a convolution layer and max pooling layer on the input data. The convolution layer uses 7x7 kernels, which is larger than we've seen in previous models. The increased kernel size is specifically for the ImageNet dataset, which has larger images than our previous datasets.

The convolution layer uses a stride size of 2 for dimension reduction. The max pooling layer also uses a stride size of 2. In total, the height and width dimensions of the input are each reduced by a factor of 4 after the initial convolution and pooling layers. This reduces computational cost, since the input data for the first block layer is now 16 times smaller.

B. Final layers

The ResNet model ends with global average pooling and a fully-connected layer to obtain the logits. The fully-connected layer is the final weight layer of the model. So the weight layers of ResNet are the initial convolution layer, the final fully-connected layer, and the convolution layers within each of the blocks.

The number of weight layers in the model gives each variation of ResNet its name (e.g. ResNet-18, ResNet-50). For example, ResNet-18 has 2 blocks in each of its 4 block layers. The blocks are regular blocks, so they have 2 convolution layers. So ResNet-18 has 16 convolution layers across all its blocks, which gives it a total of 18 weight layers.

In the next chapter you'll see the full code for the ResNet model architecture

(including the block layers and logits layer).

Full Model Architecture

Learn about the full model architecture of ResNet.

Chapter Goals:

- Learn about the full model architecture of ResNet

A. Regularization

For extremely deep models like ResNet, it is vital that we *regularize* the model, i.e. apply techniques to prevent overfitting. In the **CNN** and **SqueezeNet** sections, we used *dropout* for regularization. However, in ResNet we don't use dropout because it is normally not necessary to use dropout together with batch normalization.

The creators of batch normalization found that, along with reducing internal covariate shift, batch normalization also regularizes the model. This is because we take into account the entire batch of inputs when performing batch normalization, which reduces the chances that the model overfits on a few outliers. Since we apply batch normalization before nearly every weight layer, it alone is sufficient for regularization.

B. Increased filters

As always, we increase the number of filters used for deeper layers in the model. In ResNet, the number of filters is doubled as we go from one block layer to the next.

We increase the number of filters at block layers, rather than at individual blocks, due to the large number of blocks in ResNet. Increasing the number of filters at each block would lead to an incredibly large number of parameters, slowing down training and greatly increasing the model's memory usage.

C. Logits

The final part of the ResNet model architecture is the layer used to obtain

logits.

For the SqueezeNet model, we used a convolution layer to make the number of channels equal to the number of image classes. Then we applied global average pooling across the channels to obtain the logits.

However, while the CIFAR-10 dataset had only 10 image classes, the ImageNet dataset has 1000. Using a convolution layer with 1000 filters would require many weight parameters.

So to save weight parameters in our ResNet model, we first apply global average pooling across the channels, rather than using a convolution layer. Then we flatten the data and use a fully-connected layer to obtain the logits.

Below, we show the full code for the ResNet model architecture (using the functions from previous chapters as helpers):

```
import tensorflow as tf

class ResNetModel(object):
    # __init__ and other functions omitted

    # Model Layers
    # inputs (channels_last): [batch_size, resize_dim, resize_dim, 3]
    # inputs (channels_first): [batch_size, 3, resize_dim, resize_dim]
    def model_layers(self, inputs, is_training):
        # initial convolution layer
        conv_initial = self.custom_conv2d(
            inputs, self.filters_initial, 7, 2, name='conv_initial')
        # pooling layer
        curr_layer = tf.layers.max_pooling2d(
            conv_initial, 3, 2, padding='same',
            data_format=self.data_format,
            name='pool_initial')
        # stack the block layers
        for i, num_blocks in enumerate(self.block_layer_sizes):
            filters = self.filters_initial * 2**i
            strides = self.block_strides[i]
            # stack this block layer on the previous one
            curr_layer = self.block_layer(
                curr_layer, filters, strides,
                num_blocks, is_training, i)
        # pre-activation
        pre_activated_final = self.pre_activation(curr_layer, is_training)
        filter_size = int(pre_activated_final.shape[2])
        # final pooling layer
        avg_pool = tf.layers.average_pooling2d(
            pre_activated_final,
            filter_size,
            1,
            data_format=self.data_format)
        final_layer = tf.layers.flatten(avg_pool)
```

```
        # get logits from final layer
        logits = tf.layers.dense(final_layer, self.output_size, name='logits')
        return logits
```

D. Example image classifier

To play around with a powerful image classification tool, click this link. This model allows you to upload an image, and it will generate sets of tags relevant to that image.

Quiz

1

What problem does batch normalization solve?

COMPLETED 0%

1 of 4

Course Conclusion

A. Closing thoughts

Image recognition is one of the most essential use cases of machine learning right now, and is inherently relevant to any task involving images or videos. From healthcare and agriculture to manufacturing and retail, many companies across a variety of different industries are leveraging these technologies to get ahead.

B. Course recap

This course has provided an overview of how to perform image recognition from data processing to model predictions. You'll be able to take the practical lessons and actionable insights from this course and apply them to your projects.

C. Topics covered

- Image processing
- Convolutional neural networks
- Image classification
- SqueezeNet
- ResNet

D. Industry standard frameworks used in this course

- NumPy
- Pillow
- TensorFlow